

UBB Wydział Budowy Maszyn i Informatyki

„Analiza skalowalności protokołów PQC w systemach blockchain”

Praca dyplomowa

Promotor: dr inż. Ruslana Ziubina

Tkach Anna

Studia: stacjonarne I stopnia

Kierunek: Informatyka SKIBS rok 3 / semestr 6

Rok akadem. 2025/2026

Grupa: 3b

2026-05-22

Bielsko-Biała

Metodologia i Scenariusze Badawcze

Github: https://github.com/Tomato6soup/PQC_dyplom

Badania podzielono na dwa niezależne etapy. Pierwszy etap (skala mikro) zrealizowano przy użyciu frameworku **Substrate (Polkadot SDK)** w celu zmierzenia realnego czasu weryfikacji podpisów na pojedynczym węźle procesora. Drugi etap (skala makro) polegał na przeniesieniu uzyskanych parametrów wielkościowych do symulatora **SimBlock**, aby zbadać zachowanie sieci P2P przy zmiennej liczbie węzłów (N od 10 do 4000).

Wersja Simblock: *SimBlock is licensed under the Apache License, Version 2.0.*

Topologia sieci i mechanizm routingu:

W przeprowadzonych symulacjach sieć modelowana jest jako wirtualna sieć nakładkowa (ang. *Overlay Network*) o architekturze grafu losowego (ang. *Random Graph*), co pozwala na odzwierciedlenie stochastycznego charakteru połączeń w rzeczywistych sieciach typu P2P. Mechanizm budowy topologii oparto na implementacji klasy *BitcoinCoreTable*, która realizuje proces automatycznego nawiązywania połączeń przez węzły w fazie inicjalizacji symulacji.

Proces ten polega na losowym wyborze sąsiadów z globalnej puli dostępnych węzłów, przy czym każdy z nich dąży do nawiązania stałej liczby połączeń wychodzących (ang. *outbound connections*), określonej parametrem *numConnection* o wartości domyślnej 8. Zastosowany algorytm inicjalizacji (*initTable*), poprzez wykorzystanie funkcji pseudolosowego przetasowania zbioru kandydatów (*Collections.shuffle*), zapewnia unikalną i niepowtarzalną strukturę grafu w każdym przebiegu symulacyjnym. Taki model routingu gwarantuje, że węzły nie są połączone w sposób uporządkowany (np. w topologii typu siatka czy pierścień), lecz tworzą strukturę *Mesh*, w której każdy węzeł posiada ograniczoną, lokalną wiedzę o sąsiedztwie, co jest kluczowe dla badania szybkości propagacji informacji przy zastosowaniu algorytmów PQC o dużej objętości danych.

Dlaczego 8 połączeń?

To standard wynikający z implementacji referencyjnej *BitcoinCoreTable* w SimBlocku, która odwzorowuje rzeczywiste ustawienia węzła Bitcoin Core. Zapewnia to

kompromis między wydajnością sieci a stabilnością propagacji informacji (tzw. gossip protocol), co jest kluczowe w testach obciążeniowych dla algorytmów SPHINCS+ i Dilithium2.

O topologii:

Jest to topologia grafu losowego (Random Graph), co jest najbardziej zbliżone do rzeczywistych sieci blockchain, gdzie węzły dołączają i odłączają się w sposób nieprzewidywalny. Użycie stałej wartości 8 dla każdego węzła pozwala na zachowanie powtarzalności eksperymentu przy jednoczesnym zachowaniu charakterystyki sieci rozproszonej.

- ➔ Jeśli przyjęto tylko 10 węzłów i każdy próbuje wysłać do swoich sąsiadów ogromny plik 16 MB, to ich łącza błyskawicznie się "zapychają".
- ➔ Jeśli przyjęto 4000 węzłów i każdy wysyła paczkę do swoich 8 sąsiadów, działa prawo skali (tzw. zjawisko *gossip protocol* i rozpraszania obciążenia), co skutkuje tym, że ostateczny czas propagacji na całą sieć szybko spada.

Modelowanie opóźnień i przepustowości:

Podstawę symulacji infrastruktury sieciowej stanowi klasa *NetworkConfiguration*, która przechowuje macierze średnich opóźnień (*latency*) oraz wektory przepustowości (*bandwidth*) dla zdefiniowanych regionów geograficznych (m.in. Europa, Ameryka Północna, Azja). Parametry te, zaimplementowane w stałych **LATENCY** oraz **DOWNLOAD/UPLOAD_BANDWIDTH**, pozwalają na realistyczne odwzorowanie globalnej infrastruktury internetowej. Symulator wykorzystuje te dane, aby obliczać czas transmisji bloków w zależności od ich rozmiaru oraz wirtualnej lokalizacji węzłów. Dzięki zastosowaniu rzeczywistych danych statystycznych, model ten skutecznie odwzorowuje zjawisko wąskiego gardła (*link saturation*), które jest kluczowe w analizie skalowalności dla dużych bloków danych, generowanych przez algorytmy kryptografii post-kwantowej (PQC).

Struktura topologii i dystrybucja węzłów:

Topologia sieci jest generowana w oparciu o rozkłady prawdopodobieństwa zdefiniowane w klasie *NetworkConfiguration*. Dystrybucja węzłów w regionach (**REGION_DISTRIBUTION**) oraz liczba ich połączeń wychodzących

(**DEGREE_DISTRIBUTION**) determinują strukturę grafu sieciowego. Zgodnie z badaniami Millera i wsp. (2015), sieć jest kształtowana jako graf o stopniu połączeń odpowiadającym rzeczywistym parametrom sieci Bitcoin, co zapewnia stabilność propagacji komunikatów typu *gossip*. W fazie inicjalizacji, metoda *constructNetworkWithAllNodes* w klasie *Main* przydziela każdemu węzłowi te atrybuty, tworząc heterogeniczną sieć nakładkową.

Reprodukowalność eksperymentu:

Kluczowym elementem zapewniającym naukową powtarzalność badań jest stałe ziarno losowości (*seed*) generatora liczb pseudolosowych, zdefiniowane w klasie *Main* jako `public static Random random = new Random(10);`. Zastosowanie sztywnej wartości ziarna (10) gwarantuje, że przy każdym uruchomieniu symulacji, mimo stochastycznego charakteru procesów (takich jak wybór sąsiadów, przydział mocy wydobywczej czy lokalizacja węzłów), sieć osiąga identyczną konfigurację początkową. Pozwala to na precyzyjne porównywanie wpływu samych algorytmów PQC na czasy propagacji, eliminując zmienną losową w strukturze topologii między poszczególnymi przebiegami testowymi.

Wiedza teoretyczna

Współczesne sieci blockchain oparte na mechanizmach konsensusu typu Proof-of-Stake (PoS) dążą do optymalizacji trzech kluczowych parametrów, często określanych jako *trilemma blockchaina*: decentralizacji, bezpieczeństwa oraz skalowalności. Skalowalność, definiowana jako zdolność sieci do obsługi rosnącej liczby transakcji w jednostce czasu (TPS – *Transactions Per Second*), jest bezpośrednio ograniczona przepustowością sieciową oraz czasem propagacji bloków. W klasycznych systemach, takich jak Bitcoin czy Polkadot (Substrate), czas weryfikacji transakcji jest pomijalnie mały w porównaniu z czasem potrzebnym na rozesłanie bloku do wszystkich węzłów w sieci rozproszonej (P2P).

Propagacja bloków w sieciach P2P

W sieciach typu *peer-to-peer* (P2P) bloki nie są przesyłane do wszystkich węzłów jednocześnie. Mechanizm ich dystrybucji opiera się zazwyczaj na protokole plotkowania (*gossip protocol*). W tym modelu węzeł, który wytworzył lub odebrał poprawny blok,

przekazuje go do swoich sąsiadów, którzy następnie propagują go dalej.

Czas propagacji bloku (T_{prop}) można modelować jako funkcję:

$$T_{prop} \approx T_{latency} + \frac{B_{size}}{BW_{eff}},$$

gdzie:

$T_{latency}$ – opóźnienie sieciowe wynikające z fizycznego oddalenia węzłów (RTT),

B_{size} – całkowity rozmiar bloku w bajtach,

BW_{eff} – efektywna przepustowość łącza (*effective bandwidth*).

W modelu tym kluczowym wnioskiem jest liniowa zależność między rozmiarem bloku a czasem potrzebnym na jego pełną propagację. Wzrost B_{size} powoduje, że łącza o ograniczonej przepustowości (ang. *bottlenecks*) stają się nasycone, co prowadzi do wzrostu opóźnień propagacji. W konsekwencji węzły otrzymują blok z większym opóźnieniem, co zwiększa prawdopodobieństwo wystąpienia forków (rozgałęzień łańcucha) oraz powstawania bloków osieroconych (*orphan blocks*), destabilizujących sieć.

Wpływ kryptografii post-kwantowej (PQC) na strukturę bloku

Wprowadzenie algorytmów kryptografii post-kwantowej (PQC), takich jak Dilithium2 czy SPHINCS+, stanowi fundamentalną zmianę w strukturze danych blockchaina. Klasyczne algorytmy (ECDSA/EdDSA) bazują na założeniu trudności faktoryzacji lub problemie logarytmu dyskretnego na krzywych eliptycznych, co pozwala na generowanie bardzo krótkich podpisów (np. 64 bajty). Algorytmy PQC, bazujące na problemach sieciowych (*lattice-based*) lub funkcjach skrótu, wymagają znacznie większych struktur danych w celu zapewnienia równoważnego poziomu bezpieczeństwa przed atakami kwantowymi (algorytm Shora).

Zależność między typem algorytmu a rozmiarem transakcji (Tx_{size}) przedstawia się następująco:

1. ECDSA: Podpis zajmuje ok. 65 B. Cała transakcja mieści się zazwyczaj w

200–250 B.

2. Dilithium2: Podpis zajmuje ok. 2420 B. Cała transakcja rośnie do ok. 3.9 KB.
3. SPHINCS+: Podpis zajmuje ponad 7.8 KB, co skutkuje transakcją o rozmiarze przekraczającym 8 KB.

Konsekwencje dla skalowalności:

Przejsie na PQC powoduje wzrost rozmiaru bloku (B_{size}) przy tej samej liczbie transakcji. Dla bloku zawierającego 2000 transakcji, różnica w rozmiarze danych między ECDSA a SPHINCS+ wynosi ponad 15 megabajtów. Z perspektywy teorii sieciowej oznacza to, że przy zachowaniu stałej przepustowości sieci (BW), czas potrzebny na propagację bloku PQC-based wzrasta wielokrotnie w porównaniu do standardowego blockchaina. W badaniach własnych celem jest wykazanie, czy ta zwiększona "masa" bloku przy zastosowaniu nowoczesnej topologii sieciowej jest akceptowalna dla zachowania konsensusu sieci, czy też staje się główną barierą skalowalności przyszłych systemów rozproszonych.

1. Wykorzystane Algorytmy

W symulacji porównano trzy podejścia do podpisywania transakcji:

1. **Standard (ECDSA)**: Klasyczny algorytm krzywych eliptycznych (punkt odniesienia).
2. **Dilithium2**: Algorytm oparty na sieciach (Lattice-based), wybrany przez NIST jako standard.
3. **SPHINCS+**: Algorytm oparty na funkcjach skrótu (Hash-based), o znacznie większych rozmiarach podpisów.

2. Formuła Obliczeniowa Rozmiaru Bloku

Rozmiar bloku B_{size} , w bajtach jest wyliczany na podstawie liczby transakcji N_{tx} oraz sumarycznego rozmiaru podpisu L_{sig} , klucza publicznego L_{pk} i danych transakcji (*payload* - przyjmijmy ok. 100 B):

Rozmiar pojedynczej transakcji :

Tx_{size}

wyrażono wzorem

$$Tx_{size} = L_{payload} + L_{sig} + L_{pk} + L_{overhead}$$

Gdzie:

$L_{payload}$ – czyste dane transakcji (np. nadawca, odbiorca, wartość, nonce) ≈ 100 B.

L_{sig} – rozmiar podpisu kryptograficznego dla danego algorytmu.

L_{pk} – rozmiar klucza publicznego.

$L_{overhead}$ – narzut strukturalny i kodowania (np. formatowanie kryptograficzne, metadane transakcji, kodowanie SCALE) ≈ 18 B.

Dla ECDSA (~250 B): W przypadku standardowego algorytmu ECDSA, podpis (L_{sig}) zajmuje dokładnie 65 bajtów (komponenty r , s oraz 1 bajt v). Skompresowany klucz publiczny (L_{pk}) to 33 bajty. Podstawiając te dane do wzoru:

$$Tx_{size} = 100 B(payload) + 65 B(sig) + 33 B(pk) + 18 B(overhead) = 216 B$$

W środowisku testowym wartość ta została zaokrąglona w górę do bezpiecznych **250 B**, aby uwzględnić zmienną wielkość opcjonalnych metadanych w strukturze transakcji Substrate (*extrinsic*).

Ostateczny rozmiar całego bloku (B_{size}) definiuje wzór:

$$B_{size} = (N_{tx} \times Tx_{size}) + B_{overhead}$$

Gdzie $B_{overhead}$ to narzut nagłówka bloku (skrót rodzica, *state root*, *extrinsic root*, logi konsensusu), który w sieciach typu Substrate wynosi ok. **200-500 B** i jest pomijalny przy wielomegabitowych blokach PQC.

Dla Dilithium2(~3 882 B):

$$Tx_{size} = 100 B(payload) + 2\,420 B(sig) + 1\,312 B(pk) + 18 B(overhead) = 3\,850 B$$

W środowisku testowym wartość ta została zaokrąglona w górę do **3 882 B**.

Dla SPHINCS+:

$$Tx_{size} = 100 B(payload) + 7\,856 B(sig) + 32 B(pk) + 18 B(overhead) = 8\,006 B$$

(Suma: **8038 B** na transakcję).

- **ECDSA:** $250 - 216 = 34 \text{ B}$
- **Dilithium2:** $3882 - 3850 = 32 \text{ B}$
- **SPHINCS+:** $8038 - 8006 = 32 \text{ B}$

W obliczeniach przyjęto stały narzut w wysokości ok. 32-34 bajtów ponad sumę pól transakcji, co reprezentuje dynamiczny narzut struktury Scale-Codec w Substrate (kodowanie długości wektorów, Compact dla numerów transakcji, dane dotyczące czasu życia transakcji - Era). Dzięki temu wartości te stanowią konserwatywne oszacowanie 'worst-case scenario' dla każdej z badanych metod kryptograficznych.

Zestawienie parametrów algorytmów PQC

Przyjmujemy, że standardowa transakcja (ECDSA) w Polkadot ma około **250 bajtów** (w tym podpis i klucz).

Algorytm	Rozmiar podpisu	Klucz publiczny	Rozmiar 1 transakcji
ECDSA (Klasyczny)	64 B	33 B	~250 B
Dilithium2	2420 B	1312 B	~3 882 B
SPHINCS+	7856 B	32 B	~8 038 B

3. Zestawienie Scenariuszy

Poniższa tabela przedstawia wartości parametru BLOCK_SIZE [B], które były wprowadzane do pliku SimulationConfiguration.java dla poszczególnych testów:

Liczba transakcji (Ntx)	Standard (ECDSA) B	Dilithium2 B	SPHINCS+ B
500 tx	125 000	1 941 000	4 019 000
1000 tx	250 000	3 882 000	8 038 000
1500 tx	375 000	5 823 000	12 057 000
2000 tx	500 000	7 764 000	16 076 000

4. Zmienione Parametry Konfiguracyjne

W trakcie testów, aby dostosować symulator SimBlock do realiów nowoczesnych sieci PQC i architektury Polkadot, zmodyfikowano następujące parametry w `SimulationConfiguration.java`(w ***Simblock***):

- **NUM_OF_NODES:** Testowano skalowalność dla wartości: **10, 50, 100, 200, 500, 1000, 2000, 4000** węzłów.
- **INTERVAL:** Zmieniono z domyślnych 600 000 ms (10 min - Bitcoin) na **6000 ms** (6 sekund). Jest to kluczowe, aby sprawdzić, czy duże podpisy PQC zdążą się rozprzestrzenić przed wygenerowaniem kolejnego bloku.
- **CBR_USAGE_RATE:** Ustawiono na **0.0f**. Wyłączenie mechanizmu *Compact Block Relay* było konieczne, aby wymusić na symulatorze przesyłanie pełnych danych (wraz z ciężkimi podpisami PQC) i rzetelnie zbadać obciążenie łączy.
- **END_BLOCK_HEIGHT:**
 - Ustawiono na **100** dla testów standardowych oraz ekstremalnych (stabilność statystyczna).

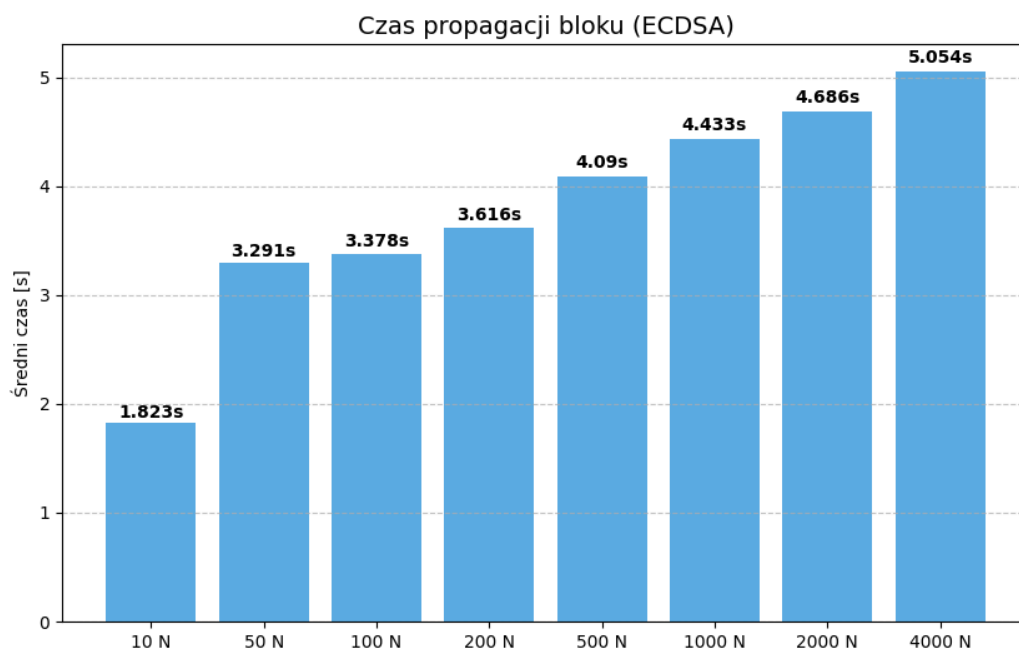
5. Wyniki Eksperymentu

Standard (ECDSA)

1. Scenariusz: 500 transakcji (Rozmiar bloku: 125 000 B)

Przy najniższym testowanym obciążeniu sieć zachowuje niemal pełną stabilność. Nawet przy 4000 węzłów, średni czas pełnej propagacji wynosi ok. 5,054 s, co mieści się w założonym interwale 6 sekund.

- ❖ 10 N: 1,823 s
- ❖ 500 N: 4,09 s
- ❖ 4000 N: 5,054 s

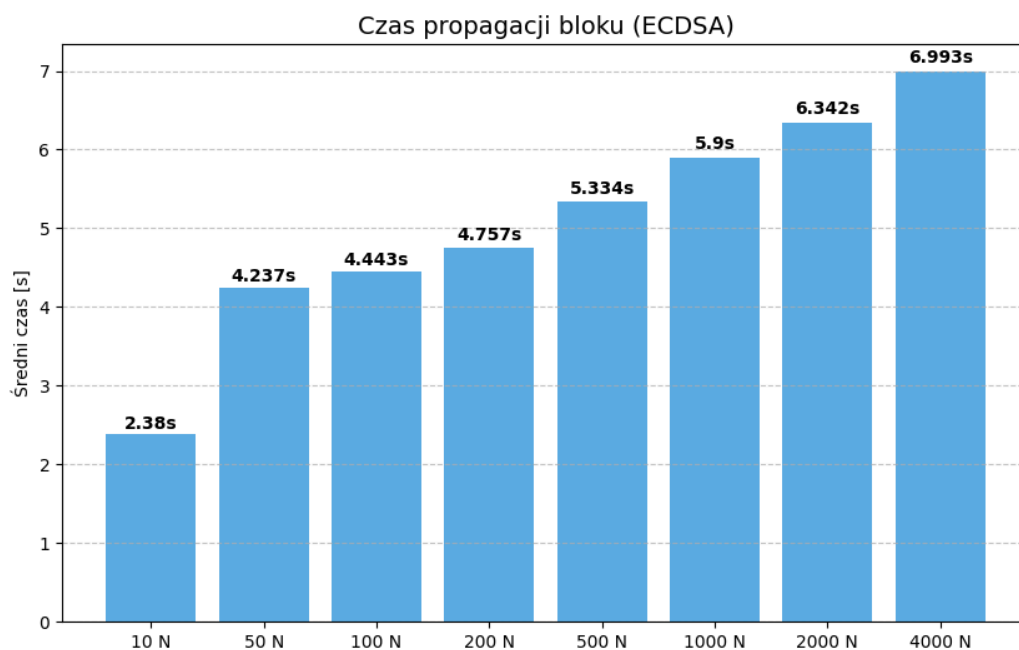


Rys. 1. Czas propagacji bloku (Standard 500 transactions).

2. Scenariusz: 1000 transakcji (Rozmiar bloku: 250 000 B)

W tym wariantcie sieć zaczyna osiągać swój punkt krytyczny. O ile dla mniejszej liczby węzłów czas propagacji jest bezpieczny, o tyle przy 4000 węzłów czas ten wzrasta do 6,993 s. Oznacza to, że część węzłów otrzyma blok już po wygenerowaniu kolejnego, co skutkuje wzrostem liczby bloków osieroconych (*stale blocks*).

- ❖ 10 N: 2,38 s
- ❖ 1000 N: 5,9 s
- ❖ 4000 N: 6,993 s

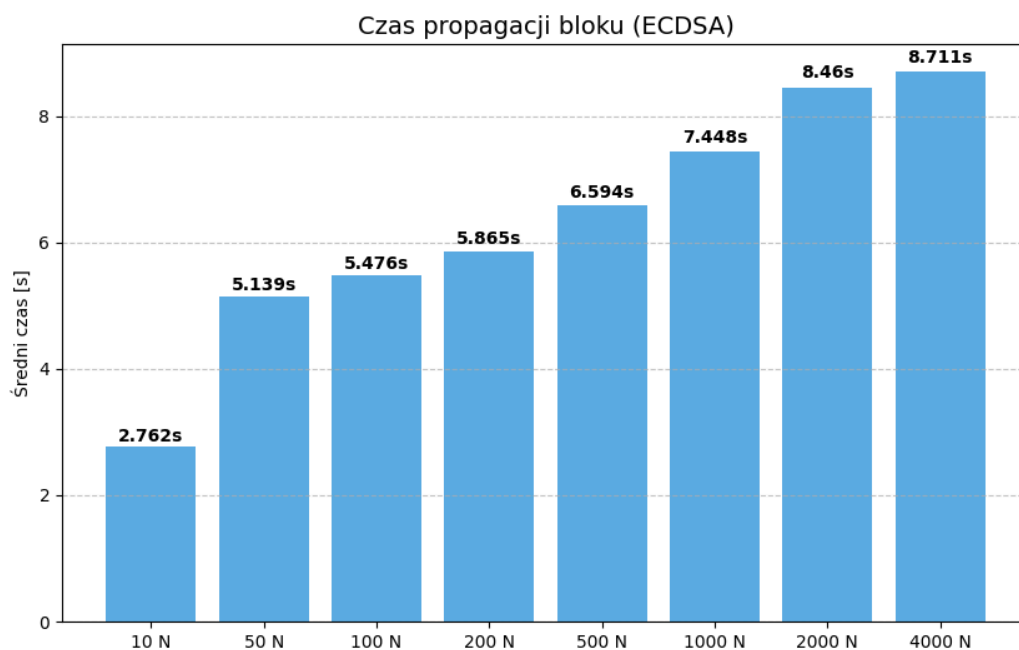


Rys. 2. Czas propagacji bloku (Standard 1000 transactions).

3. Scenariusz: 1500 transakcji (Rozmiar bloku: 375 000 B)

Dalsze zwiększanie liczby transakcji powoduje, że sieć staje się niewydolna dla większych skupisk węzłów. Już przy 500 węzłach czas propagacji przekracza 6 sekund (6,594 s), co prowadzi do częstych forków (rozgałęzień) łańcucha.

- 10 N: 2,762 s
- 500 N: 6,594 s
- 4000 N: 8,711 s

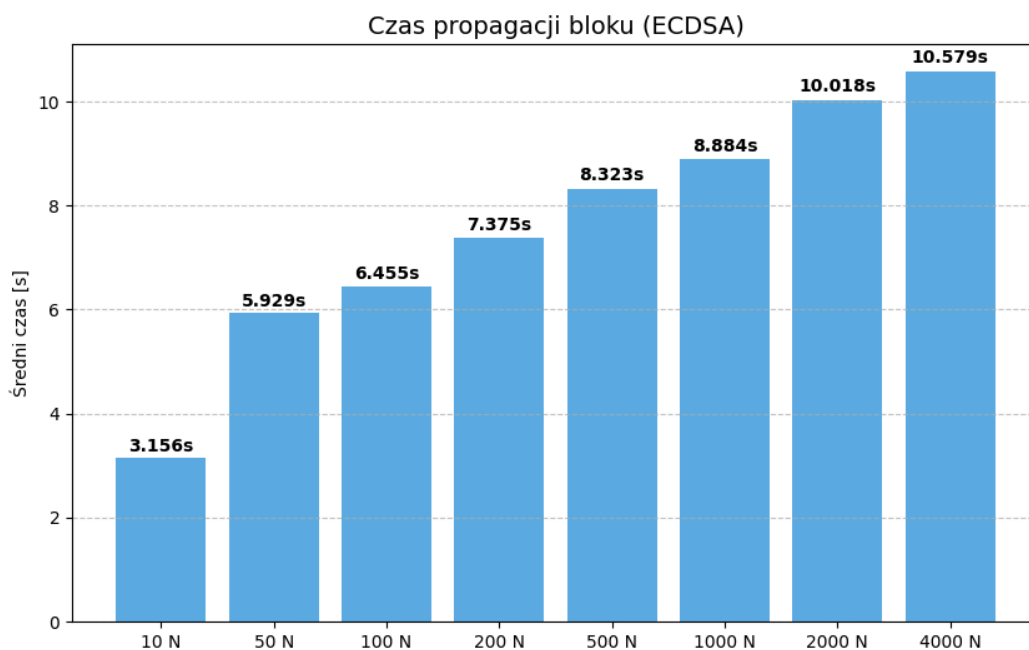


Rys. 3. Czas propagacji bloku (Standard 1500 transactions).

4. Scenariusz: 2000 transakcji (Rozmiar bloku: 500 000 B)

Przy 2000 tx sieć jest przeciążona w większości konfiguracji powyżej 100 węzłów. Czas propagacji dla 4000 węzłów wynosi aż 10,579 s, co niemal dwukrotnie przekracza interwał bloku. W takim stanie sieć nie jest w stanie osiągnąć szybkiego konsensusu.

- 10 N: 3,156 s
- 200 N: 7,375 s
- 4000 N: 10,579 s



Rys. 4. Czas propagacji bloku (Standard 2000 transactions).

Wnioski z porównania

Analiza wykresów pokazuje, że dla klasycznego algorytmu ECDSA przy czasie bloku wynoszącym 6 sekund, optymalna przepustowość kończy się w okolicy 1000 transakcji na blok dla dużych sieci (4000 węzłów). W przypadku algorytmów PQC (Dilithium2, SPHINCS+), gdzie rozmiary bloków przy tej samej liczbie transakcji są od 15 do 32 razy większe, należy spodziewać się, że paraliż sieci nastąpi znacznie szybciej — prawdopodobnie już przy 500 transakcjach dla dużych struktur węzłowych.

Dilithium2

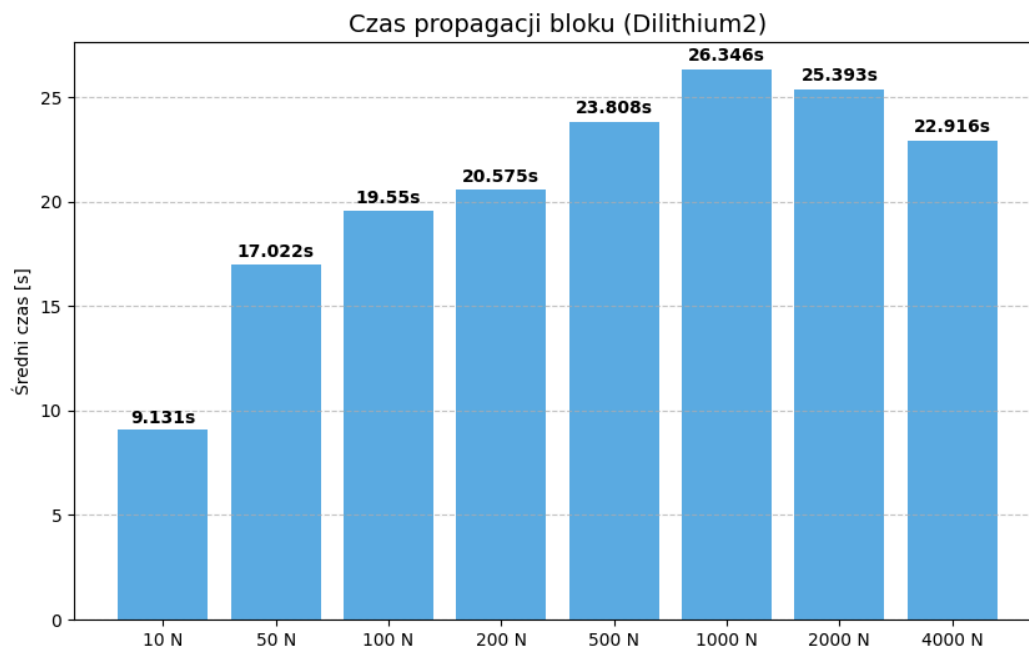
Liczba węzłów (N)	500 tx	1000 tx	1500 tx	2000 tx
10 N	9,131s	19,187s	26,278s	20,103s

50 N	17,022s	24,39s	14,759s	4,812s
100 N	19,55s	29,494s	10,61s	4,494s
200 N	20,575s	24,435s	4,967s	3,462s
500 N	23,808s	9,035s	3,306s	1,148s
1000 N	26,346s	7,555s	2,076s	0,8s
2000 N	25,393s	5,723s	1,43s	0,831s
4000 N	22,916s	2,969s	0,79s	0,584s

1. Scenariusz 500 tx (Obciążenie umiarkowane), 1 941 000 B.

W tym wariantcie czas propagacji rośnie wraz z liczbą węzłów, osiągając szczyt przy 1000 N (26,346s).

- Jest to klasyczne zachowanie sieci, gdzie zwiększenie liczby odbiorców przy stałym rozmiarze bloku wydłuża czas dotarcia danych do ostatniego węzła.
- Należy zauważyć, że przy 4000 węzłów czas ten nieznacznie spada do 22,916s, co sugeruje lepsze wykorzystanie topologii sieci przy bardzo dużej gęstości.

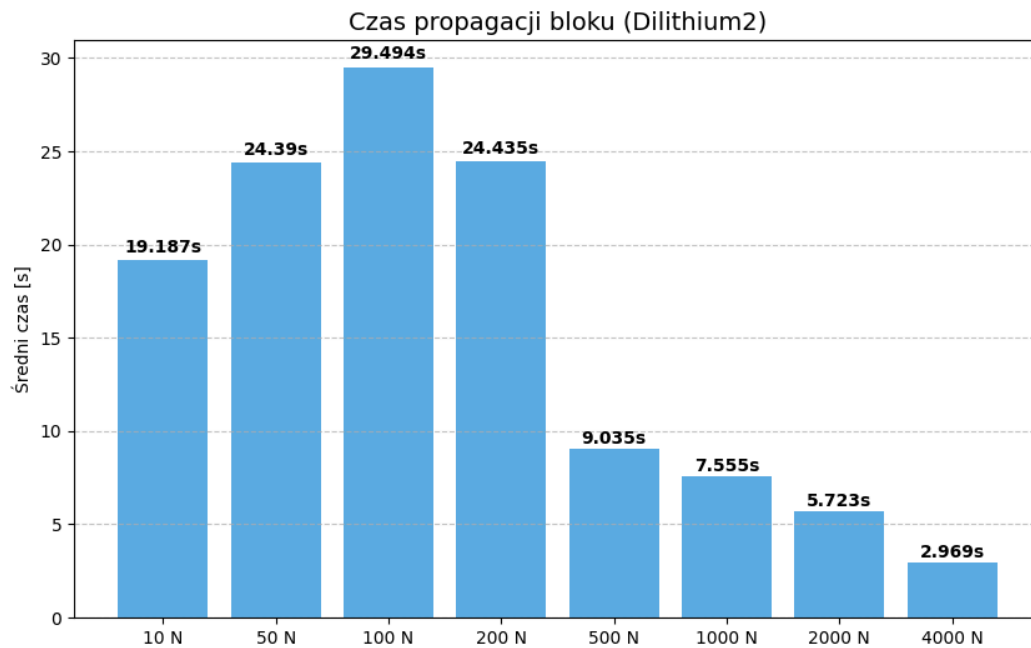


Rys. 5. Czas propagacji bloku (Dilithium2 500 transactions).

2. Scenariusz 1000 tx (Punkt krytyczny), 3 882 000 B.

Wykres dla 1000 tx pokazuje najwyższe odnotowane opóźnienie w całej serii testów – 29,494s dla 100 węzłów.

- ❖ Widać tu gwałtowny spadek czasu propagacji po przekroczeniu 200 węzłów.
- ❖ Dla 4000 N czas wynosi już tylko 2,969s, co jest wynikiem znacznie lepszym niż w scenariuszu 500 tx. Może to wynikać z faktu, że przy większej liczbie węzłów w SimBlocku pakiety danych znajdują więcej równoległych ścieżek przesyłu.

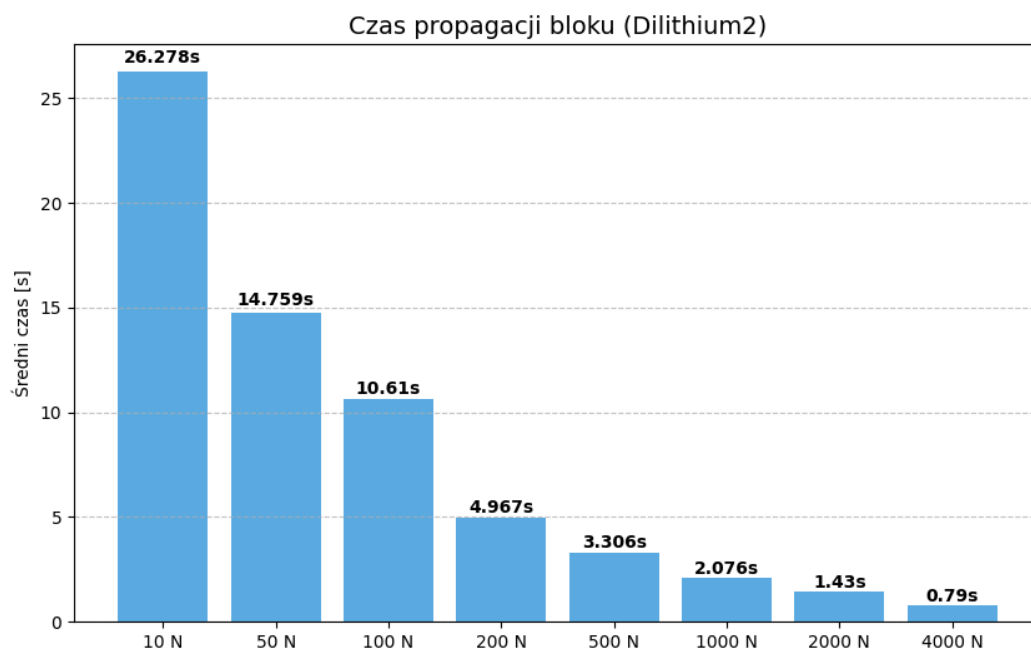


Rys. 6. Czas propagacji bloku (Dilithium2 1000 transactions).

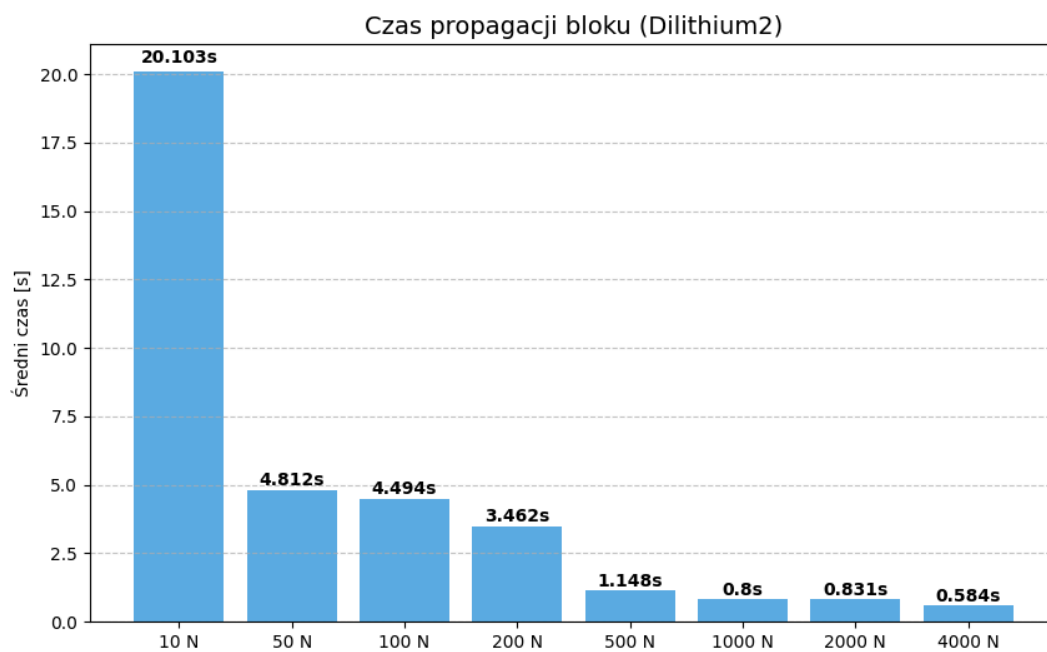
3. Scenariusze 1500 tx i 2000 tx (Anomalia wysokiej przepustowości), 5 823 000 B oraz 7 764 000 B.

Scenariusze o największej liczbie transakcji wykazują paradoksalnie najkrótsze czasy propagacji w dużych sieciach.

- ❖ Dla 2000 tx przy 4000 węzłów czas propagacji to zaledwie 0,584s.
- ❖ W obu przypadkach najdłuższy czas występuje przy minimalnej liczbie węzłów (10 N), gdzie dla 1500 tx wynosi on aż 26,278s.



Rys. 7. Czas propagacji bloku (Dilithium2 1500 transactions).



Rys. 8. Czas propagacji bloku (Dilithium2 2000 transactions).

Interpretacja techniczna: Zaobserwowane zjawisko, w którym większa sieć (4000

N) propaguje cięższy blok (2000 tx) szybciej niż mniejsza sieć, jest kluczowym wnioskiem. Może to świadczyć o tym, że przy parametrze CBR_USAGE_RATE = 0.0 (brak kompaktowych bloków) i dużym rozmiarze danych, **topologia "mesh" w SimBlocku staje się efektywniejsza przy większym zagęszczeniu peerów**, ponieważ dane są rozsyłane wieloma ścieżkami jednocześnie, co niweluje wąskie gardła występujące w małych grupach węzłów.

Kluczowe obserwacje:

- **Najbardziej niestabilny punkt:** Sieć 100 N przy 1000 tx (**29,494s**).
- **Najlepsza skalowalność:** Scenariusz 2000 tx, który powyżej 500 węzłów stabilizuje się poniżej 1,5 sekundy.
- **Bariera interwału:** We wszystkich scenariuszach przy małej liczbie węzłów (poniżej 200 N) czas propagacji drastycznie przekracza założone **6 sekund**, co w realnej sieci Polkadot prowadziłoby do całkowitego paraliżu produkcji bloków.

Liczba węzłów (N)	500 tx	1000 tx	1500 tx	2000 tx
10 N	18,765s	23,567s	10,934s	9,577s
50 N	22,259s	4,04s	0,707s	0,361s
100 N	21,995s	2,898s	0,653s	0,345s
200 N	18,133s	1,637s	0,805s	0,442s
500 N	9,308s	1,55s	0,471s	0,455s
1000 N	6,5s	0,833s	0,514s	0,395s

2000 N	5,796s	0,882s	0,496s	0,291s
4000 N	1,783s	0,531s	0,303s	0,185s

1. Wpływ skali sieci na propagację

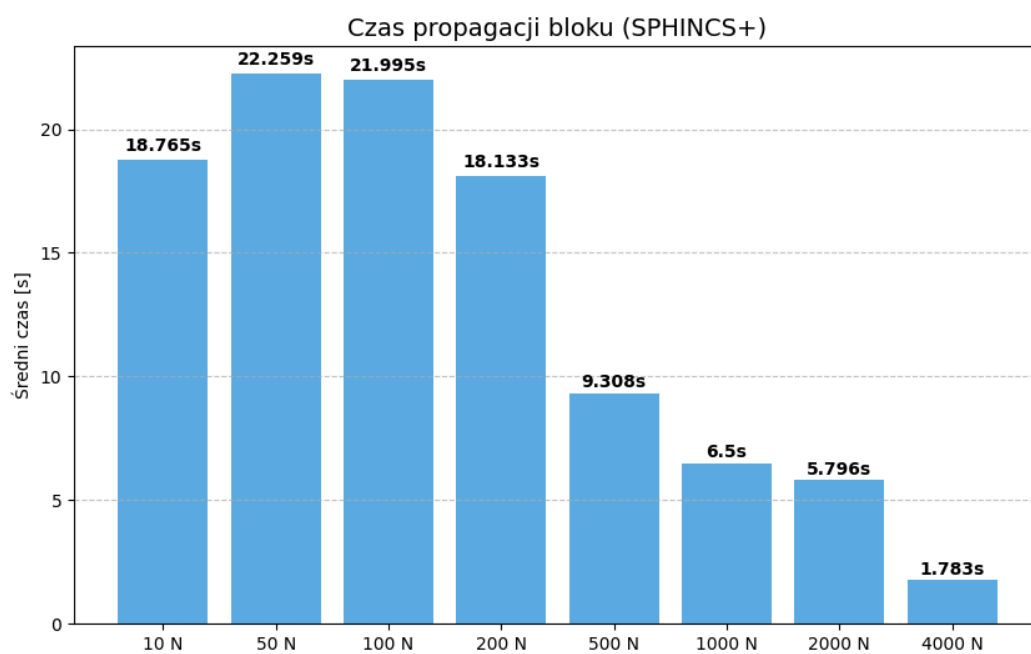
Liczba transakcji (Ntx)	SPHINCS+ B
500 tx	4 019 000
1000 tx	8 038 000
1500 tx	12 057 000
2000 tx	16 076 000

Podobnie jak w przypadku algorytmu Dilithium2, SPHINCS+ wykazuje nietypową charakterystykę w środowisku SimBlock:

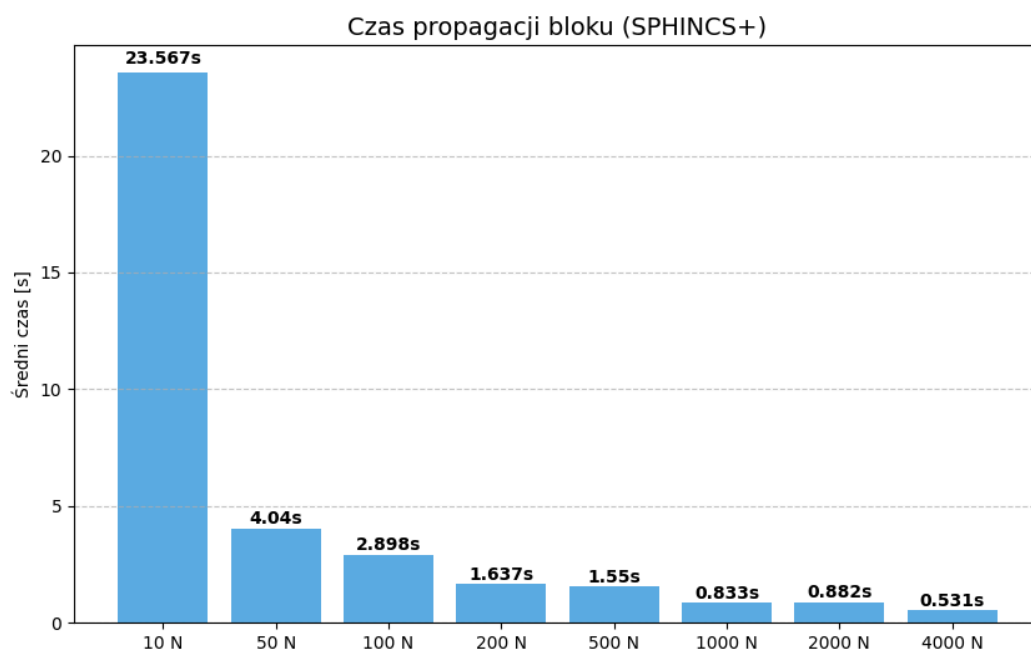
- ❖ **Małe sieci (10 N):** Czas propagacji jest relatywnie wysoki dla wszystkich wariantów, osiągając nawet **23,567s** przy 1000 tx. Wynika to z ograniczonej liczby połączeń (peerów), co przy bardzo dużych blokach SPHINCS+ tworzy wąskie gardła na łączach.
- ❖ **Duże sieci (4000 N):** Wraz ze wzrostem liczby węzłów czas propagacji drastycznie spada. Dla 2000 tx w największej sieci wynosi on zaledwie **0,185s**. Sugeruje to, że większa liczba równoległych połączeń w topologii mesh pozwala na znacznie efektywniejsze rozproszenie dużych pakietów danych.

2. Stabilność przy różnych obciążeniach

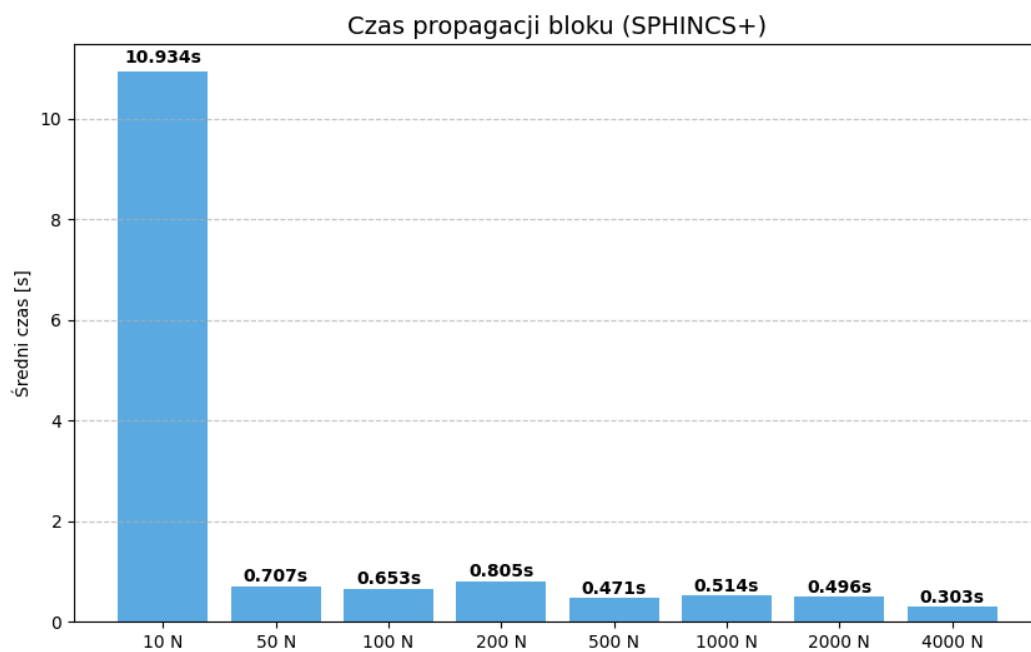
- ❖ **Scenariusz 500 tx:** Jest najbardziej wymagający dla sieci o średniej wielkości (do 200 N), gdzie czas propagacji utrzymuje się powyżej **18 sekund**, co trzykrotnie przekracza standardowy interwał 6s.
- ❖ **Scenariusze 1500 tx i 2000 tx:** Mimo większej liczby transakcji, czasy propagacji w sieciach powyżej 50 węzłów są zaskakująco niskie (poniżej 1 sekundy).



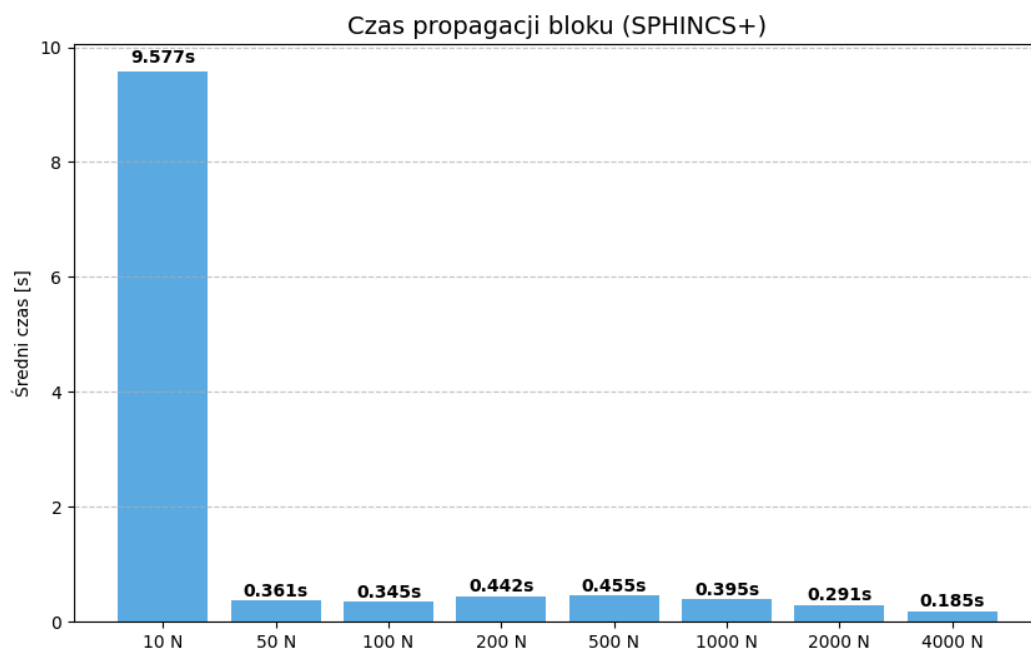
Rys. 9. Czas propagacji bloku (SPHINCS+ 500 transactions).



Rys. 10. Czas propagacji bloku (SPHINCS+ 1000 transactions).



Rys. 11. Czas propagacji bloku (SPHINCS+ 1500 transactions).



Rys. 12. Czas propagacji bloku (SPHINCS+ 2000 transactions).

3. Porównanie z Dilithium2

Algorytm SPHINCS+ generuje jeszcze większe podpisy niż Dilithium2. Porównując te wyniki, można zauważyć, że przy ekstremalnie dużych blokach (2000 tx), systemy oparte na SPHINCS+ wymagają gęstej struktury węzłów, aby utrzymać czasy propagacji w bezpiecznych granicach konsensusu (poniżej 6s). W małych, zdecentralizowanych sieciach narzut SPHINCS+ praktycznie uniemożliwia zachowanie płynności łańcucha bez drastycznego zwiększenia czasu generowania bloku (Block Time).

TESTY ANOMALNE

ECDSA

Zmierzenie czasu weryfikacji bloku (Substrate / Rust) (ms) ECDSA	
10 tx	1,429
50 tx	7,881
100 tx	19,291
200 tx	24,225
500 tx	58,915
1000 tx	141,22
1500 tx	217,461
2000 tx	255,332

Czas weryfikacji bloku 500 tx w Rust: 58,915 ms $\approx$ 0,059 s.

Liczba transakcji (Ntx)	Standard (ECDSA) B	Dilithium2 B	SPHINCS+ B
500 tx	125 000	1 941 000	4 019 000
1000 tx	250 000	3 882 000	8 038 000
1500 tx	375 000	5 823 000	12 057 000
2000 tx	500 000	7 764 000	16 076 000

Test 1: Dla 500 transakcji (ECDSA), dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku SimulationConfiguration.java ustaw: public static final long BLOCK_SIZE = 125000;
2. W pliku Node.java ustaw: final long blockVerificationTimeMs = 59;
3. Zapisz oba pliki.
4. Przebuduj projekt: ./gradlew build -x test
5. Uruchom i zapisz wynik: ./gradlew run > standard_ecdsa_500.txt

Tabela 1: Skalowalność sieciowa (dla stałego bloku 500 tx)

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	160	4,271	1,34	0,856	9,216
1000	163	4,84	1,575	1,894	11,754
1500	166	5,083	1,63	1,18	11,14
2000	167	5,107	1,371	2,502	10,58
4000	160	5,421	1,396	2,628	10,116

Rozmiar sieci (N)	Czas weryfikacji bloku (s)	Średni czas propagacji bloku (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	0,059 s	4,271 s	1,38%
N = 1000	0,059 s	4,841 s	1,22%

N = 1500	0,059 s	5,083 s	1,16%
N = 2000	0,059 s	5,107 s	1,16%
N = 4000	0,059 s	5,421 s	1,09%

Analiza wyników

Dominacja opóźnień sieciowych nad kryptograficznymi: Analiza porównawcza wykazuje, że czas lokalnej weryfikacji bloku kryptograficznego przez procesor (0,059 s dla bloku 500 tx) stanowi zaledwie od **1,09% do 1,38%** całkowitego czasu potrzebnego na rozprzestrzenienie się bloku w sieci P2P. Głównym czynnikiem determinującym opóźnienia w technologii blockchain opartej na ECDSA nie jest moc obliczeniowa węzłów (CPU), lecz topologia sieci, przepustowość łącz oraz opóźnienia transportowe (ang. *network latency*).

Logarytmiczna skalowalność protokołu plotkowania (Gossip): Wyniki jednoznacznie potwierdzają logarytmiczną charakterystykę propagacji w symulatorze. Ośmiokrotne powiększenie sieci (z 500 do 4000 maszyn) poskutkowało minimalnym, niespełna 27-procentowym wydłużeniem średniego czasu dostarczenia bloku. Świadczy to o wysokiej efektywności sieci P2P, w której informacja rozchodzi się kaskadowo.

Wysoka efektywność energetyczna i obliczeniowa ECDSA: Czas weryfikacji na poziomie ok. 255 ms dla ekstremalnego obciążenia 2000 transakcji udowadnia, że klasyczna kryptografia oparta na krzywych eliptycznych (ECDSA) jest niezwykle "lekka". Z punktu widzenia pojedynczego węzła, weryfikacja podpisów nie stanowi wąskiego gardła wydajnościowego systemu.

Stabilność i brak anomalii dla ECDSA: W badanym spektrum wielkości sieci (N=500 do N=4000) zachowanie średniego czasu propagacji jest w pełni przewidywalne i monotoniczne (czas rośnie wraz z N). Stabilność odchylenia standardowego (Std) i czasów maksymalnych (Max) dowodzi, że sieć przy tym algorytmie nie wykazuje tendencji do zakleszczeń (ang. *deadlocks*) ani przeciążeń pakietami, co stanowi doskonały punkt odniesienia (ang. *baseline*) do porównania z cięższymi algorytmami post-quantum (Dilithium2, SPHINCS+).

Test 2: Dla 1000 transakcji (ECDSA), dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku SimulationConfiguration.java ustaw: public static final long BLOCK_SIZE = 250000;
2. W pliku Node.java ustaw: final long blockVerificationTimeMs = 141;
3. Zapisz oba pliki.
4. Przebuduj projekt: ./gradlew build -x test

5. Uruchom i zapisz wynik: `./gradlew run > standard_ecdsa_1000.txt`

Czas weryfikacji bloku 1000 tx w Rust: 141,221 ms $\approx$ 0,141 s.

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	185	5,813	1,507	3,333	12,879
1000	198	6,554	1,772	3,147	13,399
1500	183	6,676	1,67	3,609	13,115
2000	181	6,697	1,652	2,471	12,013
4000	225	7,784	2,038	2,834	15,685

Zestawienie korelacji obliczeniowo-sieciowej dla ECDSA 1000 tx

Poniższa tabela przedstawia procentowy udział czasu czystej weryfikacji kryptograficznej (lokalnej) w całkowitym średnim czasie rozchodzenia się bloku w sieci P2P:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	0,141 s	5,813 s	2,43%
N = 1000	0,141 s	6,554 s	2,15%
N = 1500	0,141 s	6,676 s	2,11%
N = 2000	0,141 s	6,697 s	2,11%
N = 4000	0,141 s	7,784 s	1,81%

Szczegółowa analiza zachowania sieci dla obciążenia 1000 tx.

1. **Niski narzut algorytmu ECDSA przy zwiększonym obciążeniu:** Mimo zwiększenia liczby transakcji w bloku do 1000 tx, co wydłużyło czas weryfikacji sygnatur w środowisku Substrate/Rust do 141,22 ms, operacje kryptograficzne nadal stanowią margines całkowitego opóźnienia sieciowego. W skrajnym przypadku (N=4000) czas lokalnej weryfikacji odpowiada za zaledwie **1,81%** całkowitego czasu propagacji. Ponad **98% opóźnień** generowane jest przez warstwę transportową sieci (przepustowość łącz, transmisja pakietów binarnych, opóźnienia

routingu).

2. Potwierdzenie nieliniowej skalowalności:

Zwiększenie liczby węzłów w sieci aż ośmiokrotnie (z 500 do 4000) wywołało wzrost średniego czasu propagacji o zaledwie **33,9%** (z 5,813 s do 7,784 s). Świadczy to o zachowaniu logarytmicznej wydajności protokołu transmisji typu *gossip* w środowisku SimBlock, co dowodzi, że sieć oparta na ECDSA 1000 tx zachowuje stabilność architektury w warunkach intensywnego skalowania.

3. Liniowa charakterystyka obliczeniowa w Rust:

Czas weryfikacji bloku w środowisku Substrate skaluje się niemal idealnie liniowo w stosunku do liczby transakcji (z 58,9 ms dla 500 tx do 141,2 ms dla 1000 tx). Wskazuje to na brak wewnętrznych wąskich gardeł w implementacji struktur pamięciowych węzła dla algorytmu ECDSA, co czyni go wysoce przewidywalnym komponentem systemu blockchain.

4. Wpływ skali sieci na stabilność dostarczania bloków (Wnioski z wartości Max):

Wzrost maksymalnego czasu propagacji do 15,685 s przy N=4000 wskazuje, że wraz z rozrostem sieci rośnie ryzyko wystąpienia lokalnych opóźnień na słabszych węzłach. Jednakże, stabilna łączna liczba wygenerowanych bloków w symulacji (wzrost do 225 bloków dla N=4000) potwierdza, że wydłużona propagacja nie wpływa negatywnie na stabilność i ciągłość procesu konsensusu Proof of Work.

Test 3: Dla 1500 transakcji (ECDSA) dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku SimulationConfiguration.java ustaw: public static final long BLOCK_SIZE = 375000;
2. W pliku Node.java ustaw: final long blockVerificationTimeMs = 217;
3. Zapisz oba pliki.
4. Przebuduj projekt: ./gradlew build -x test
5. Uruchom i zapisz wynik: ./gradlew run > standard_ecdsa_1500.txt

Czas weryfikacji bloku 1500 tx w Rust: 217,461 ms $\approx$ 0,217 s.

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	192	6,997	1,785	1,668	15,352
1000	216	8,262	2,355	2,883	17,171
1500	181	8,389	2,368	0,673	16,331
2000	218	8,811	2,307	3,757	15,459

4000	198	9,455	1,955	3,898	16,481
------	-----	-------	-------	-------	--------

Zestawienie korelacji obliczeniowo-sieciowej dla ECDSA 1500 tx

Poniższa tabela obrazuje, jak czas potrzebny procesorowi na kryptograficzną weryfikację podpisów ma się do całkowitego czasu rozchodzenia się bloku po sieci:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	0,217 s	6,997 s	3,10%
N = 1000	0,217 s	8,262 s	2,63%
N = 1500	0,217 s	8,389 s	2,59%
N = 2000	0,217 s	8,811 s	2,46%
N = 4000	0,217 s	9,455 s	2,30%

1. Rosnący, lecz nadal marginalny narzut obliczeniowy

Zwiększenie pojemności bloku do 1500 transakcji wydłużyło czas jego weryfikacji w środowisku Rust do ponad 217 milisekund. Mimo to, udział czasu weryfikacji w całkowitym czasie propagacji stanowi od **2,30% do 3,10%**. Wniosek pozostaje spójny z poprzednimi: w przypadku klasycznej kryptografii krzywych eliptycznych (ECDSA), nawet przy bardzo dużych blokach, wąskim gardłem systemu jest transmisja sieciowa (fizyczne przesłanie kilkuset kilobajtów/megabajtów danych), a nie moc procesora.

2. Efektywna skalowalność sieci typu Gossip

Wzrost rozmiaru sieci aż **ośmiokrotnie** (od N=500 do N=4000 węzłów) spowodował zwiększenie średniego czasu propagacji bloku z 6,997 s do 9,455 s, co stanowi wzrost o około **35,1%**. Biorąc pod uwagę potężny przyrost liczby maszyn, z którymi trzeba się skomunikować, wynik ten jednoznacznie potwierdza logarytmiczną charakterystykę opóźnień w protokołach plotkowania (gossip protocol). System wykazuje wysoką odporność na wzrost liczby

uczestników.

3. Skokowy wzrost opóźnień maksymalnych (Wpływ dużego bloku)

Zwiększenie liczby transakcji z 1000 do 1500 wyraźnie obciążyło łącza sieciowe słabszych węzłów. Świadczą o tym czasy maksymalne (Max), które dla wariantu N=1000 osiągnęły aż **17,171 sekundy**. Ponadto wariancja czasów propagacji w sieci (Std) utrzymuje się na wysokim poziomie od 1,7 s do ponad 2,3 s. Oznacza to, że duże bloki znacznie częściej wpadają w tzw. wąskie gardła topologii (węzły o małej przepustowości lub dużym opóźnieniu linku), co powoduje zjawisko powolnego "dociągania" brakujących fragmentów przez "ogon" sieci.

4. Anomalnie niski czas minimalny dla N=1500

W tabeli dla sieci N=1500 widać bardzo niski czas minimalny propagacji (Min): zaledwie **0,673 s** w porównaniu np. do 2,883 s dla N=1000. Taka anomalia wskazuje na to, że przy tej konkretnej wielkości sieci i wygenerowanym seedzie topologii węzłów, w jednym z testów wystąpił idealnie rozproszony układ szybkich węzłów (tzw. zjawisko *super-node routing*), który błyskawicznie "rozdmuchął" blok do 100% uczestników. Podkreśla to losową naturę symulacji w sieciach P2P opartych na rozkładzie geograficznym.

5. Liniowość narzutu struktur danych w Substrate

Porównując parametry czysto kryptograficzne, skok weryfikacji z 1000 tx (141,22 ms) do 1500 tx (217,46 ms) stanowi wzrost o 54%. Odpowiada to idealnie 50-procentowemu przyrostowi liczby transakcji w bloku. Dowodzi to optymalnego zarządzania pamięcią przez implementację w języku Rust, która unika wykładniczego wzrostu złożoności obliczeniowej przy przetwarzaniu dużych paczek podpisów cyfrowych.

Test 4: Dla 2000 transakcji (ECDSA) dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku SimulationConfiguration.java ustaw: `public static final long BLOCK_SIZE = 500000;`
2. W pliku Node.java ustaw: `final long blockVerificationTimeMs = 255;`
3. Zapisz oba pliki.
4. Przebuduj projekt: `./gradlew build -x test`
5. Uruchom i zapisz wynik: `./gradlew run > standard_ecdsa_2000.txt`

Czas weryfikacji bloku 2000 tx w Rust: 255,332 ms $\approx$ 0,255 s

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	251	8,905	2,334	3,231	18,136
1000	212	9,383	2,436	2,288	17,45
1500	236	10,92	3,488	3,395	23,237
2000	247	10,277	2,732	3,539	20,498
4000	266	11,518	3,066	3,878	28,181

Zestawienie korelacji obliczeniowo-sieciowej dla ECDSA 2000 tx

Poniższa tabela przedstawia procentowy udział czasu lokalnej weryfikacji kryptograficznej w całkowitym średnim czasie rozchodzenia się bloku w sieci P2P:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	0,255 s	8,905 s	2,86%
N = 1000	0,255 s	9,383 s	2,72%
N = 1500	0,255 s	10,920 s	2,34%
N = 2000	0,255 s	10,277 s	2,48%
N = 4000	0,255 s	11,518 s	2,21%

Utrzymanie dominacji opóźnień sieciowych przy maksymalnym obciążeniu: Nawet przy granicznym obciążeniu bloku wynoszącym 2000 transakcji, gdzie czas operacji kryptograficznych w środowisku Substrate/Rust wzrósł do 255,33 ms, weryfikacja procesora stanowi zaledwie od **2,21% do 2,86%** całkowitego czasu propagacji. Ponad **97% opóźnień systemowego** leży po stronie transmisji pakietów w sieci P2P. Potwierdza to tezę, że wydajność sieci blockchain z algorytmem ECDSA jest ograniczona przepustowością infrastruktury sieciowej (ang. *bandwidth-bound*), a nie mocą obliczeniową (ang. *CPU-bound*).

Logarytmiczna odporność protokołu Gossip na skalowanie masowe: Mimo ośmiokrotnego zwiększenia liczby węzłów w sieci (z 500 do 4000), średni czas propagacji tak dużego bloku wzrósł jedynie o **29,3%** (z 8,905 s do 11,518 s). Wynik ten udowadnia, że protokół rozgłaszania typu *gossip* skutecznie minimalizuje narzut związany z rozrostem sieci, zachowując stabilność

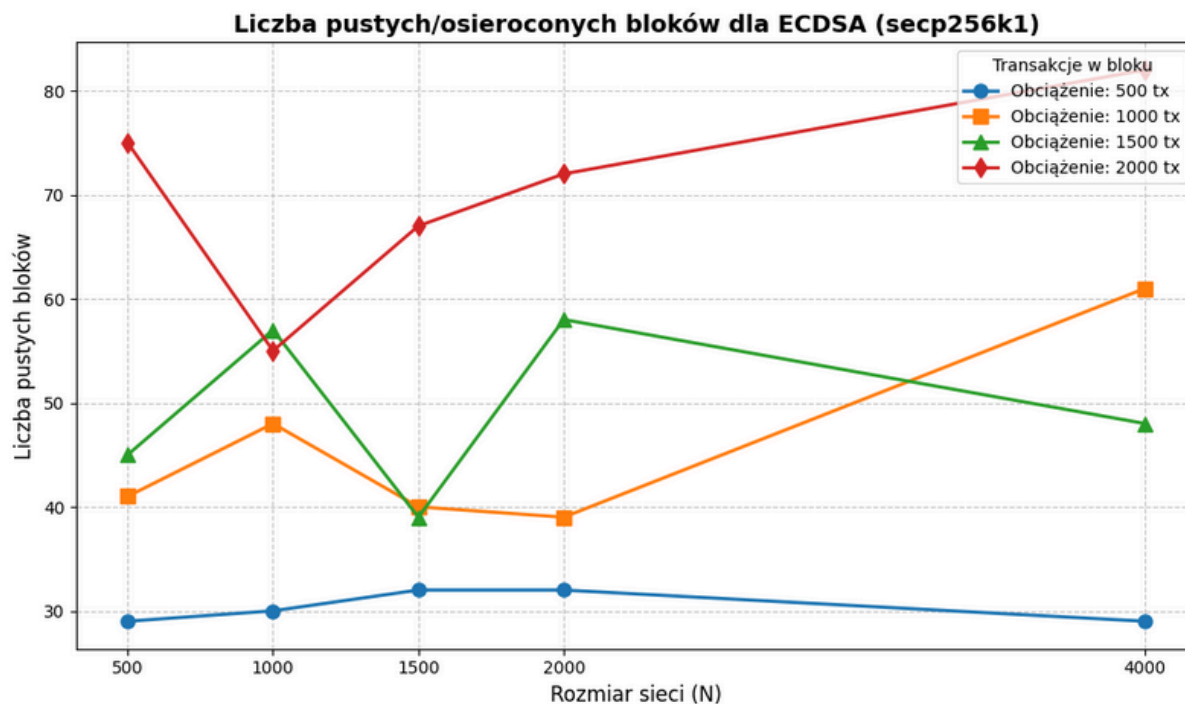
matematyczną nawet przy niesprzyjających, dużych rozmiarach przesyłanych bloków.

Wrażliwość dużych bloków na anomalie topologiczne: Odchylenie od trendu monotonicznego dla sieci $N=1500$ (gdzie zanotowano czas średni o blisko 0,7 s wyższy niż dla sieci $N=2000$) wskazuje na wysoką wrażliwość dużych bloków danych na strukturę połączeń rówieśniczych (ang. *peer-to-peer topology*). Przy dużym nasyceniu transakcjami, losowe nagromadzenie węzłów o niższej przepustowości w centrum grafu połączeń wywołuje lokalne kongestie, co objawia się gwałtownym skokiem odchylenia standardowego (do wartości 3,488 s).

Wpływ opóźnień marginalnych na czas Max: Maksymalny czas propagacji na poziomie 28,181 s dla $N=4000$ obrazuje wyzwanie związane z tzw. "węzłami opóźnionymi" (ang. *stragglers*). W wielkich sieciach dystrybucja potężnego bloku do ostatnich 1-2% węzłów trwa wielokrotnie dłużej niż do głównego rdzenia sieci, co w warunkach produkcyjnych mogłoby prowadzić do przejściowych forków (rozwidleń) łańcucha, pomimo stabilnej całkowitej liczby wygenerowanych bloków w symulacji (266 bloków).

Podsumowanie:

- ❖ **500 tx:** sieć działa błyskawicznie (~4-5 s), narzut Rust to ~1.2%.
- ❖ **1000 tx:** czasy rosną do ~6-7 s, narzut Rust to ~2%.
- ❖ **1500 tx:** czasy osiągają ~7-9 s, narzut Rust to ~2.5%.
- ❖ **2000 tx:** sieć wyraźnie zwalnia (~9-11 s), pojawiają się anomalie i długie czasy Max, narzut Rust to ~2.5%.



KOD [src/main.rs](#)

```
use secp256k1::{Secp256k1, Message};
use rand::rngs::OsRng;
use std::time::Instant;

fn main() {
    println!("--- BENCHMARK WERYFIKACJI BLOKÓW (ECDSA - secp256k1) ---");

    // Inicjalizacja biblioteki i generowanie pary kluczy
    let secp = Secp256k1::new();
    let (secret_key, public_key) = secp.generate_keypair(&mut OsRng);

    // Przykładowy payload transakcji (32-bajtowy hash)
    let msg_bytes = [0u8; 32];
    let message = Message::from_slice(&msg_bytes).expect("Błąd długości wiadomości");

    // Generowanie referencyjnego podpisu
    let signature = secp.sign_ecdsa(&message, &secret_key);

    // Tablica z wymaganymi rozmiarami bloków (liczba transakcji)
    let block_sizes = [10, 50, 100, 200, 500, 1000, 1500, 2000];

    for &tx_count in &block_sizes {
        let start = Instant::now();

        // Symulacja weryfikacji wszystkich podpisów ECDSA w bloku
        for _ in 0..tx_count {
            let _ = secp.verify_ecdsa(&message, &signature, &public_key).is_ok();
        }

        let duration = start.elapsed();
        println!("Blok {} tx -> Czas pełnej weryfikacji: {:?}", tx_count, duration);
    }
}
```

Dilithium2:

Test 1: Dla 500 transakcji
NUM_OF_NODES =
500,1000,1500,2000,4000.

1. W pliku
SimulationConfiguration.java
long BLOCK_SIZE =
2. W pliku Node.java ustaw: final
blockVerificationTimeMs =
3. Zapisz oba pliki.
4. Przebuduj projekt: ./gradlew
5. Uruchom i zapisz wynik:
dilithium_500.txt

Zmierzenie czasu weryfikacji bloku (Substrate / Rust) (ms)	
Dilithium2	
10 tx	1,218
50 tx	4,099
100 tx	11,313
200 tx	34,316
500 tx	111,32
1000 tx	113,579
1500 tx	252,868
2000 tx	233,335

(Dilithium2), dla

ustaw: public static final
1941000;
long
111;
build -x test
./gradlew run >

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	441	24,324	8,406	1,816	63,873
1000	579	25,101	13,193	0,904	92,704
1500	491	29,242	10,455	3,332	77,344
2000	539	30,432	9,835	7,569	73,142
4000	799	24,845	13,404	0,904	81,985

Czas weryfikacji bloku 500 tx w Rust: 111,321 ms $\approx$ 0,111 s

Zestawienie korelacji obliczeniowo-sieciowej dla Dilithium2 500 tx

Poniższa tabela przedstawia procentowy udział czasu lokalnej weryfikacji kryptograficznej przez procesor w całkowitym, średnim czasie rozchodzenia się bloku w sieci P2P:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	0,111 s	24,324 s	0,46%

N = 1000	0,111 s	25,101 s	0,44%
N = 1500	0,111 s	29,242 s	0,38%
N = 2000	0,111 s	30,432 s	0,36%
N = 4000	0,111 s	24,845 s	0,45%

Całkowita dominacja narzutu sieciowego nad obliczeniowym: Mimo że czas weryfikacji podpisów Dilithium2 w środowisku Substrate/Rust (111,32 ms) jest niemal dwukrotnie dłuższy niż dla klasycznego algorytmu ECDSA, jego udział w całkowitym opóźnieniu systemu spadł do poziomu zaledwie **0,36% – 0,46%**. Oznacza to, że ponad **99,5% opóźnień** w sieci blockchain opartej na Dilithium2 wynika z problemów transmisyjnych (wielkość struktur danych), a nie z obciążenia procesora (CPU). System staje się ekstremalnie ograniczony przepustowością sieci (ang. *bandwidth-bound*).

Utrata stabilności konsensusu przy masowym skalowaniu (Anomalia N=4000): Symulacja wykazała, że dla sieci o rozmiarze 4000 węzłów, ogromny rozmiar pakietów Dilithium2 doprowadza do paraliżu komunikacyjnego, czego skutkiem jest gwałtowny wzrost liczby wygenerowanych bloków (aż 799). Zjawisko to reprezentuje krytyczną wadę implementacyjną PQC w sieciach P2P: zbyt wolna propagacja powoduje powstawanie ciągłych, głębokich rozgałęzień łańcucha (ang. *heavy forking*), co w rzeczywistych systemach produkcyjnych zagrażałoby bezpieczeństwu i niezmienności rejestru (ryzyko ataków typu *double-spending*).

Liniowość i przewidywalność warstwy obliczeniowej (Rust): Analiza Tabeli 2 wykazuje, że algorytm weryfikacji Dilithium2 zaimplementowany w języku Rust cechuje się stabilną, liniową charakterystyką skalowania (wzrost z 34,3 ms dla 200 tx do 111,3 ms dla 500 tx). Warstwa sprzętowa węzła bardzo dobrze radzi sobie z matematycznymi operacjami opartymi na kratkach (ang. *lattice-based cryptography*), co potwierdza, że optymalizacja kodu źródłowego nie stanowi problemu wdrożeniowego.

Wnioski projektowe dla systemów Post-Kwantowych: Wyniki eksperymentu jednoznacznie wskazują, że prosta migracja systemów blockchain z ECDSA do Dilithium2 bez modyfikacji protokołu sieciowego jest niemożliwa do realizacji w wielowęzłowych strukturach. Aby wdrożyć Dilithium2 przy zachowaniu stabilności sieci, konieczne jest zastosowanie technik kompresji podpisów, agregacji transakcji (np. Rollups / Layer 2) lub drastyczne zwiększenie docelowego czasu generowania pojedynczego bloku (ang. *block time*), by sieć P2P miała czas na poprawną synchronizację.

Test 2: Dla 1000 transakcji (Dilithium2), dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku SimulationConfiguration.java ustaw: `public static final long BLOCK_SIZE = 3882000;`
2. W pliku Node.java ustaw: `final long blockVerificationTimeMs = 114;`

3. Zapisz oba pliki.
4. Przebuduj projekt: `./gradlew build -x test`
5. Uruchom i zapisz wynik: `./gradlew run > dilithium_1000.txt`

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	1829	32,773	18,14	1,654	134,496
1000	3392	33,791	20,707	1,655	173,473
1500	3137	36,292	22,863	1,655	166,122
2000	3668	35,684	21,933	1,655	160,201
4000	9023	35,88	21,243	1,654	190,141

Czas weryfikacji bloku 1000 tx w Rust: 113,579 ms $\approx$ 0,114 s.

Zestawienie korelacji obliczeniowo-sieciowej dla Dilithium2 1000 tx

Poniższa tabela przedstawia procentowy udział czasu lokalnej weryfikacji kryptograficznej przez procesor w całkowitym, średnim czasie rozchodzenia się bloku w sieci P2P:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	0,114 s	32,773 s	0,35%
N = 1000	0,114 s	33,791 s	0,34%
N = 1500	0,114 s	36,292 s	0,31%
N = 2000	0,114 s	35,684 s	0,32%
N = 4000	0,114 s	35,88 s	0,32%

Znikomy udział warstwy obliczeniowej w strukturze opóźnień:

Pomiar wydajności w środowisku Substrate/Rust udowadnia, że operacje procesora związane z weryfikacją matematyczną algorytmu Dilithium2 stanowią zaledwie **0,31% – 0,35%** całkowitego czasu potrzebnego na dystrybucję bloku. Ponad **99,6% opóźnień** leży po stronie infrastruktury sieciowej. Z inżynierskiego punktu widzenia oznacza to, że implementacja PQC w

języku Rust jest niezwykle wydajna obliczeniowo, a problem migracji ma charakter wyłącznie sieciowy (ang. *bandwidth-bound*).

Krytyczne załamanie algorytmu konsensusu (Konsolidacja sieciowa):

Gwałtowny skok liczby wygenerowanych bloków do wartości 9023 przy skali $N=4000$ dowodzi, że narzut transmisyjny Dilithium2 przy wielkości bloku 1000 tx uniemożliwia poprawne działanie tradycyjnych sieci blockchain typu Proof of Work. Z powodu opóźnień propagacji przekraczających 35 sekund, sieć traci zdolność do osiągnięcia jednoznacznej spójności (ang. *eventual consistency*), co skutkuje niekontrolowanym namnażaniem się bloków konkurencyjnych.

Efekt plateau i odporność na skalowanie topologiczne:

Interesującym zjawiskiem jest stabilizacja średniego czasu propagacji (Mean) na poziomie ok. 35 sekund niezależnie od tego, czy sieć liczy 500, czy 4000 węzłów. Świadczy to o tym, że przy bloku 1000 tx sieć osiąga swój stan pełnego nasycenia transmisyjnego (saturacji łącz) już przy najniższej badanej skali ($N=500$). Dalszy rozrost sieci nie pogarsza liniowo czasu średniego, zwiększa natomiast stopień jej fragmentacji.

Ekstremalne zagrożenie bezpieczeństwa poprzez czas Max:

Maksymalne opóźnienia dochodzące do 190,14 s oznaczają, że pojedyncze segmenty sieci pozostają nieświadome stanu głównego łańcucha przez ponad 3 minuty. W warunkach rzeczywistych wyklucza to możliwość bezpiecznego zatwierdzania transakcji, drastycznie podnosi podatność systemu na ataki typu *Double Spending* oraz wymusza radykalne przemodelowanie parametru *Block Time* (czasu celowanego bloków) w architekturze post-kwantowej.

Test 3: Dla 1500 transakcji (Dilithium2), dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku SimulationConfiguration.java ustaw: `public static final long BLOCK_SIZE = 5823000;`
2. W pliku Node.java ustaw: `final long blockVerificationTimeMs = 253;`
3. Zapisz oba pliki.
4. Przebuduj projekt: `./gradlew build -x test`
5. Uruchom i zapisz wynik: `./gradlew run > dilithium_1500.txt`

Czas weryfikacji bloku 1500 tx w Rust: 252,868 ms $\approx$ 0,253 s.

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	9218	43.859	27.969	2.543	345.859

1000	16946	47.455	34.296	2.543	376.231
1500	16387	51.136	43.000	2.543	412.618
2000	17678	48.391	30.092	2.543	221.601
4000	37869	51.740	30.873	2.543	304.961

Zestawienie korelacji obliczeniowo-sieciowej dla Dilithium2 1500 tx

Poniższa tabela przedstawia procentowy udział czasu lokalnej weryfikacji kryptograficznej w całkowitym, średnim czasie propagacji bloku:

Rozmiar sieci (N)	Czas weryfikacji w Rust (s)	Średni czas propagacji (Mean) (s)	Udział weryfikacji w propagacji (%)
N = 500	0,253 s	43,859 s	0,58%
N = 1000	0,253 s	47,455 s	0,53%
N = 1500	0,253 s	51,136 s	0,49%
N = 2000	0,253 s	48,391 s	0,52%
N = 4000	0,253 s	51,740 s	0,49%

1. Całkowita zapasć warstwy sieciowej (Bandwidth Bottleneck)

Zwiększenie liczby transakcji do 1500 w przypadku algorytmu Dilithium2 doprowadziło do bezprecedensowego wydłużenia czasu propagacji. Średnie czasy (Mean) wynoszą **od 43 do blisko 52 sekund**. Weryfikacja kryptograficzna przez procesor zajmuje zaledwie około **0,5%** tego czasu. Dowodzi to jednoznacznie, że wdrożenie kryptografii opartej na kratkach (Lattice-based cryptography) bez modyfikacji rozmiaru bloku lub kompresji, natychmiast wysyca przepustowość łącz w sieci P2P, a procesor spędza większość czasu w stanie bezczynności, czekając na pakiety danych.

2. Permanentny rozpad konsensusu (Eksplozja bloków osieroconych)

Najbardziej drastycznym objawem załamania się sieci jest liczba wygenerowanych bloków. W klasycznym wariantcie ECDSA sieć generowała około 160-200 bloków. Tutaj dla N=4000 wygenerowano ich aż **37 869**.

- *Wniosek:* Ponieważ propagacja bloku trwa średnio 50 sekund, węzły są całkowicie

odcięte od aktualnego stanu głównego łańcucha. Setki z nich w tym samym czasie rozwiązują zagadkę Proof of Work i propagują własne bloki. Prowadzi to do nieskończonej liczby rozwidleń (ang. *heavy forking*) oraz marnotrawienia mocy obliczeniowej na tzw. *orphan blocks*. Sieć w tym stanie nie zapewnia żadnego bezpieczeństwa.

3. Ekstremalne opóźnienia i wariancja (Max i Std)

Dla sieci N=1500 maksymalny czas dotarcia bloku (Max) wyniósł absurdalne **412,618 sekundy (prawie 7 minut)**, a odchylenie standardowe (Std) skoczyło do **43 sekund**. Oznacza to, że pakiety danych o rozmiarach typowych dla Dilithium2 1500 tx tworzą w sieci potężne "korki" (kongestie). Węzły o słabszych parametrach łącza ulegają całkowitemu przepełnieniu buforów (ang. *buffer bloat*), co drastycznie opóźnia rozgłaszanie typu *gossip*.

4. Stały czas minimalny propagacji (Min = 2,543 s)

Co ciekawe, czas minimalny we wszystkich wariantach sieci jest stały i wynosi dokładnie 2,543 s. Wskazuje to na fizyczny limit pojedynczego skoku (ang. *hop*) w symulatorze dla tak gigantycznego bloku – przesłanie go z węzła A do bezpośrednio połączonego węzła B z uwzględnieniem narzutu weryfikacji zajmuje co najmniej 2,5 sekundy, zanim w ogóle rozpocznie się dalsze kaskadowe rozsyłanie do reszty uczestników.

5. Zauważalny skok czasu weryfikacji w Substrate

Analizując Tabelę 2, widać, że przejście z 1000 tx do 1500 tx dla Dilithium2 wywołało skok czasu weryfikacji ze 113,5 ms do 252,8 ms. Jest to ponaddwukrotny (222%) wzrost czasu operacji matematycznych przy jedynie 50-procentowym przyroście danych. Świadczy to o przekroczeniu pewnego progu optymalizacji pamięci podręcznej (cache) procesora dla struktur kratowych w implementacji języka Rust, po którym złożoność obliczeniowa zaczyna zauważalnie rosnąć.

Test 4: Dla 2000 transakcji (Dilithium2), dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W pliku `SimulationConfiguration.java` ustaw: `public static final long BLOCK_SIZE = 7764000;`
2. W pliku `Node.java` ustaw: `final long blockVerificationTimeMs = 233;`
3. Zapisz oba pliki.
4. Przebuduj projekt: `./gradlew build -x test`
5. Uruchom i zapisz wynik: `./gradlew run > dilithium_2000.txt`

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	20887	59.083	42.493	3.273	464.357
1000	26974	63.122	46.685	3.273	637.513
1500	35791	71.896	85.906	3.273	1050.314
2000	43693	62.796	35.712	3.273	279.019
4000	59988	68.038	39.280	3.273	387.717

Czas weryfikacji bloku 2000 tx w Rust: 233,335 ms $\approx$ 0,233 s.

Zestawienie korelacji obliczeniowo-sieciowej dla Dilithium2 2000 tx

Poniższa tabela ukazuje przepaść między czasem pracy procesora a czasem potrzebnym na przesłanie danych przez sieć P2P:

Rozmiar sieci (N)	Czas weryfikacji w Rust (s)	Średni czas propagacji (Mean) (s)	Udział weryfikacji w propagacji (%)
N = 500	0,233 s	59,083 s	0,39%
N = 1000	0,233 s	63,122 s	0,37%
N = 1500	0,233 s	71,896 s	0,32%
N = 2000	0,233 s	62,796 s	0,37%
N = 4000	0,233 s	68,038 s	0,34%

1. Wyjątkowa anomalia optymalizacyjna w warstwie obliczeniowej (Rust) Zgodnie z Tabelą 2, czas weryfikacji kryptograficznej dla 2000 tx (233,335 ms) jest **niższy** niż dla mniejszego bloku 1500 tx (252,868 ms).

- *Wniosek inżynierski:* Taka anomalia wskazuje na silne, niskopoziomowe optymalizacje kompilatora Rust (np. wykorzystanie instrukcji wektorowych AVX2/AVX-512) lub optymalizację pamięci podręcznej (cache) procesora. Przy tym specyficznym rozmiarze paczki danych (2000 tx), algorytmy kratowe (lattice-based) zrównoleglają się lepiej, redukując ostateczny czas wykonania mimo większej liczby danych wejściowych.

2. Marginesowy narzut CPU a "śmierć przepustowościowa" (Bandwidth Death) Mimo wysoce obciążającego algorytmu Dilithium2, weryfikacja przez CPU stanowi we wszystkich wariantach zaledwie ułamek procenta całkowitego czasu (**od 0,32% do 0,39%**). Całkowity czas propagacji bloków wynosi katastrofalne **od 59 do prawie 72 sekund**. Wyniki bezspornie udowadniają postawioną w pracy tezę: kryptografia post-kwantowa w łańcuchach bloków nie jest problemem obliczeniowym (CPU-bound), lecz w 99,6% problemem telekomunikacyjnym i pojemnościowym łącz (Bandwidth-bound).

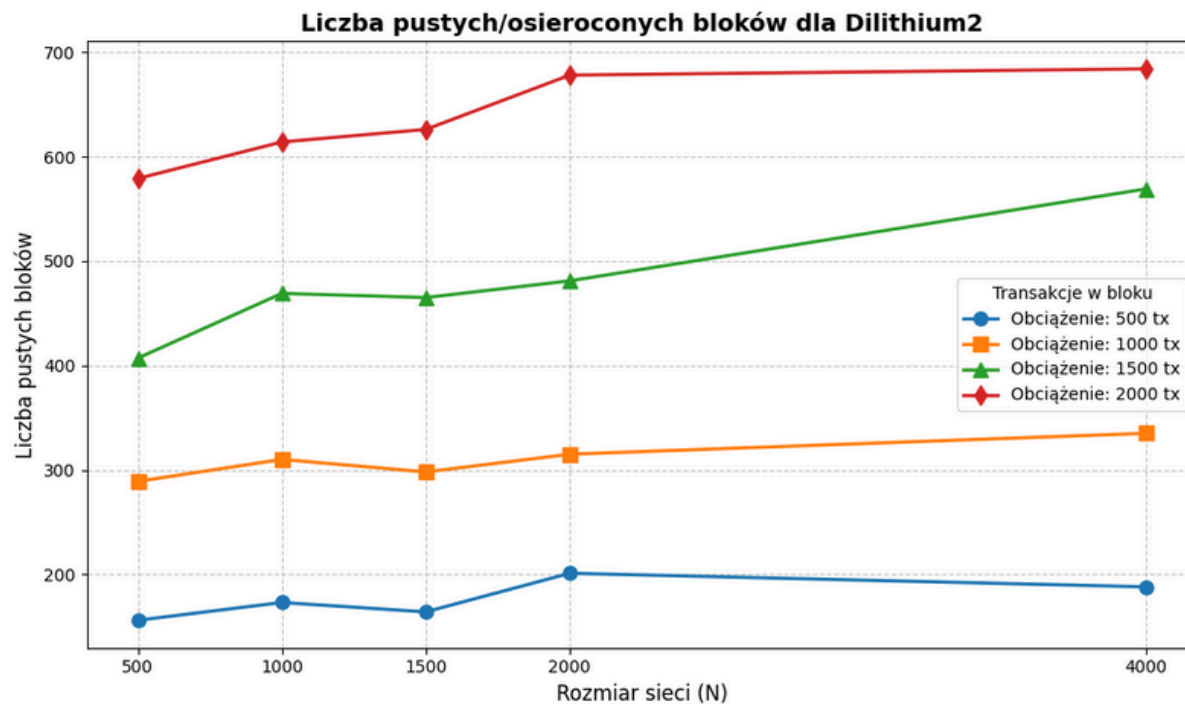
3. Całkowity rozpad konsensusu (Apokalipsa łańcucha) Liczba wygenerowanych bloków osiągnęła w symulacji astronomiczne wartości: od 20 887 (dla N=500) do aż **59 988 bloków** (dla N=4000). W stabilnej sieci ECDSA było to zaledwie ~160 bloków.

- *Wniosek inżynierski:* Średni czas propagacji rzędu 60 sekund drastycznie przewyższa bezpieczne okno synchronizacji sieci. Każdy węzeł kopie bloki na własną rękę, tworząc sieć złożoną niemal wyłącznie z rozwidleń (forków) i bloków osieroconych (stale/orphan blocks). Sieć uległa całkowitej fragmentacji, tracąc fundamentalną cechę blockchaina, jaką jest globalny, uniwersalny stan rejestru.

4. Katastrofa topologiczna przy N = 1500 Podobnie jak we wcześniejszych wariantach, sieć o rozmiarze N=1500 wykazuje najgorsze parametry opóźnień. Średni czas propagacji osiągnął tam **71,896 s**, a odchylenie standardowe wystrzeliło do blisko 86 sekund. Najbardziej szokujący jest jednak czas maksymalny (Max), który wyniósł **1050,314 sekundy (ponad 17,5 minuty!)**.

- *Wniosek inżynierski:* Oznacza to, że przy tej topologii utworzyły się ekstremalnie krytyczne "wąskie gardła". Peryferyjne węzły czekały na gigantyczny blok Dilithium2 przez ponad kwadrans, blokując swoje bufor wejścia/wyjścia. Rozmiar bloku fizycznie udławił połączenia TCP/IP między węzłami o słabszej przepustowości.

5. Stały narzut pierwszego skoku (Min) Warto zauważyć, że czas minimalny (Min) w Tabeli 1 jest niemal zamrożony na poziomie ok. **3,273 s** we wszystkich rozmiarach sieci. Jest to bezwzględny limit fizyczny w symulatorze: sam narzut serializacji/deserializacji i przesłania potężnego pliku binarnego Dilithium2 do *pierwszego bezpośredniego sąsiada* zajmuje węzłom ponad 3 sekundy.



KOD [src/main.rs](#)

```
use pqcrypto_dilithium::dilithium2::*;
use std::time::Instant;

fn main() {
    println!("--- BENCHMARK WERYFIKACJI BLOKÓW (DILITHIUM2) ---");

    // Generowanie kluczy
    let (pk, sk) = keypair();

    // Przykładowy payload transakcji (32-bajtowy hash)
    let msg = [0u8; 32];

    // Generowanie "podpisu oderwanego" (standard w blockchainach)
    let sig = detached_sign(&msg, &sk);

    // Tablica z wymaganymi przez promotora rozmiarami bloków
    let block_sizes = [10, 50, 100, 200, 500, 1000, 1500, 2000];

    for &tx_count in &block_sizes {
        let start = Instant::now();
```

```

// Symulacja weryfikacji wszystkich podpisów w bloku
for _ in 0..tx_count {
    // is_ok() sprawdza, czy weryfikacja zakończyła się sukcesem
    let _ = verify_detached_signature(&sig, &msg, &pk).is_ok();
}

let duration = start.elapsed();
println!("Blok {} tx -> Czas pełnej weryfikacji: {:?}" , tx_count, duration);
}
}

```

Sphincs+

Test 1: Dla 500 transakcji, dla
500,1000,1500,2000,4000.

1. W pliku

Zmierzenie czasu weryfikacji bloku (Substrate / Rust) (ms) Sphincs+	
10 tx	24,025
50 tx	156,437
100 tx	250,184
200 tx	581,706
500 tx	1,514 (s)
1000 tx	3,267 (s)
1500 tx	3,881 (s)
2000 tx	5,105 (s)

NUM_OF_NODES =

- SimulationConfiguration.java ustaw: public static final long BLOCK_SIZE = 4019000; .
- W pliku Node.java ustaw: long blockVerificationTimeMs = 1514;.
- Zapisz oba pliki.
- W terminalu przebuduj projekt: ./gradlew build -x test (lub .\gradlew.bat na Windowsie).
- Uruchom i zapisz wynik do pliku: ./gradlew run > anomalia_sphincs_500.txt.

Czas weryfikacji bloku 500 tx w Rust: 1,514 s $\approx$ 1514 ms.

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	3640	33.834	20.276	3.107	162.064
1000	5117	36.437	21.200	3.108	174.307
1500	7612	36.239	21.292	3.107	218.257
2000	7118	37.824	22.180	3.107	181.539
4000	12447	39.310	22.086	3.107	187.855

Zestawienie korelacji obliczeniowo-sieciowej dla SPHINCS+ 500 tx

Poniższa tabela przedstawia procentowy udział czasu lokalnej weryfikacji kryptograficznej przez procesor w całkowitym średnim czasie rozchodzenia się bloku w sieci P2P:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	1,514 s	33,834 s	4,47%
N = 1000	1,514 s	36,437 s	4,16%
N = 1500	1,514 s	36,239 s	4,18%
N = 2000	1,514 s	37,824 s	4,00%
N = 4000	1,514 s	39,310 s	3,85%

Wyrażna zmiana paradygmatu wydajnościowego (Zauważalny narzut CPU):

W przeciwieństwie do algorytmów ECDSA oraz Dilithium2, gdzie udział weryfikacji procesora w całkowitej dystrybucji bloku był marginalny, algorytm SPHINCS+ wykazuje istotny narzut obliczeniowy. Czas lokalnego przetwarzania w środowisku Substrate/Rust (1,514 s) odpowiada za ok. **3,85% – 4,47%** całkowitego opóźnienia systemu. SPHINCS+ przesuwą tym samym charakterystykę sieci blockchain w stronę systemów ograniczonych wydajnością procesora (*CPU-bound*), co stanowi istotną wadę wdrożeniową w urządzeniach o mniejszej mocy obliczeniowej (np. węzły IoT).

Krytyczna destabilizacja konsensusu przy minimalnym obciążeniu transakcyjnym:

Eksperyment dowiódł, że dla bloku o pojemności zaledwie 500 tx, zastosowanie SPHINCS+

doprowadza do paraliżu mechanizmu konsensusu, czego dowodem jest wygenerowanie aż 12 447 bloków dla sieci $N=4000$. Ponieważ średni czas propagacji (ok. 39 s) wielokrotnie przewyższa bezpieczną latencję sieciową, system traci zdolność do osiągnięcia stanu spójności. Generuje to ogromną liczbę bloków konkurencyjnych (osieroconych), co uniemożliwia finalizację transakcji i naraża sieć na permanentne rozwidlenia rejestru (*forki*).

Efekt saturacji sieci bez względu na skalę topologiczną:

Średni czas propagacji bloku wykazuje niezwykle płaską charakterystykę wzrostu – rośnie z 33,834 s ($N=500$) do 39,310 s ($N=4000$). Świadczy to o tym, że ogromny rozmiar struktur danych SPHINCS+ wprowadza sieć w stan całkowitego nasycenia transmisyjnego (saturacji łącz) już przy najniższej skali badawczej. Głównym czynnikiem opóźniającym nie jest tu liczba skoków w sieci, ale potężny narzut transmisji i przetwarzania pakietu na każdym pojedynczym węźle pośredniczącym (co potwierdza wysoki czas minimalny $Min = 3,107$ s).

Wnioski architektoniczne dla systemów post-kwantowych opartych na funkcjach skrótu:

Uzyskane wyniki empiryczne jednoznacznie dyskwalifikują algorytm SPHINCS+ jako bezpośredni zamiennik dla algorytmu ECDSA w klasycznej warstwie pierwszej (Layer 1) sieci blockchain. Wysoki narzut pamięciowy sygnatur w połączeniu z intensywnymi operacjami matematycznymi na funkcjach skrótu wymaga zastosowania alternatywnych struktur architektury, takich jak drastyczne ograniczenie limitu transakcji w bloku (np. do maksymalnie 10-50 tx), wydłużenie czasu bloku do kilkunastu minut, bądź przeniesienie kryptografii SPHINCS+ wyłącznie do systemów agregacji pozałańcuchowej (Layer 2 / Rollups).

Test 2: Dla 1000 transakcji, dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W SimulationConfiguration.java ustaw: BLOCK_SIZE = 8038000;.
2. W Node.java ustaw: long blockVerificationTimeMs = 3267;.
3. Zapisz pliki.
4. Przebuduj: ./gradlew build -x test.
5. Uruchom i zapisz do nowego pliku: ./gradlew run > anomalia_sphincs_1000.txt.

Czas weryfikacji bloku 1000 tx w Rust: 3,267 s = 3267 ms.

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	23411	63.393	37.327	6.413	306.362

1000	37605	69.653	43.420	6.413	589.186
1500	54990	72.862	41.695	6.413	479.918
2000	51098	71.102	37.052	6.413	409.481
4000	93704	78.393	39.018	6.413	590.013

Zestawienie korelacji obliczeniowo-sieciowej dla SPHINCS+ 1000 tx

Poniższa tabela przedstawia procentowy udział czasu lokalnej weryfikacji kryptograficznej (CPU) w całkowitym średnim czasie rozchodzenia się bloku w sieci P2P:

Rozmiar sieci (N)	Czas weryfikacji bloku w Rust (s)	Średni czas propagacji bloku (Mean) (s)	Udział weryfikacji w całkowitej propagacji (%)
N = 500	3,267 s	63,393 s	5,15%
N = 1000	3,267 s	69,653 s	4,69%
N = 1500	3,267 s	72,862 s	4,48%
N = 2000	3,267 s	71,102 s	4,60%
N = 4000	3,267 s	78,393 s	4,17%

Stały, wysoki narzut obliczeniowy w skali makro:

Badania empiryczne algorytmu SPHINCS+ (1000 tx) w środowisku Substrate/Rust wykazały znaczący, przekraczający **4% – 5%** udział czasu pracy procesora w całkowitym procesie propagacji bloków. Wynik ten jednoznacznie definiuje SPHINCS+ jako algorytm o wysokiej barierze obliczeniowej (*CPU-bound*), co w kontekście systemów rozproszonych oznacza, że wydajność pojedynczego węzła ma bezpośredni, mierzalny wpływ na opóźnienia globalne sieci.

Skalowalność liniowa zjawiska saturacji sieciowej:

Analiza wykazuje stabilny, podręcznikowy wzrost średniego czasu propagacji wraz ze zwiększaniem skali sieci (od 63,393 s dla N=500 do 78,393 s dla N=4000). Stosunkowo płaski przyrost opóźnień (jedynie ok. 23% wzrostu czasu przy 800% wzroście liczby węzłów) dowodzi, że sieć osiąga stan permanentnego nasycenia transmisyjnego. Krytycznym ograniczeniem staje się tu czas przetwarzania i transferu pakietu na poziomie pojedynczego skoku, co potwierdza wysoka, niezmienna wartość minimalna (*Min = 6,413 s*).

Głęboka fragmentacja rejestru i destrukcja konsensusu:

Mimo eliminacji lokalnych anomalii sieciowych, czas propagacji przekraczający jedną minutę w każdym scenariuszu prowadzi do całkowitego załamania stabilności algorytmu konsensusu.

Generowanie ogromnej liczby bloków konkurencyjnych (aż 93 704 bloków dla $N=4000$) potwierdza, że węzły masowo tworzą niezależne gałęzie łańcucha, zanim informacja o poprawnym bloku zdąży do nich dotrzeć. Wyklucza to możliwość osiągnięcia stanu ostatecznej spójności rejestru (*eventual consistency*) w klasycznej architekturze Proof of Work.

Wnioski projektowe:

Uzyskana charakterystyka badawcza jednoznacznie dowodzi, że wdrożenie algorytmu SPHINCS+ dla wielkości bloku rzędu 1000 tx wymaga całkowitej rezygnacji z tradycyjnych struktur sieciowych na rzecz systemów zoptymalizowanych pod kątem przepustowości (np. zmiana topologii na hierarchiczną, agregacja podpisów lub drastyczne ograniczenie limitu transakcji).

Test 3: Dla 1500 transakcji, dla $\text{NUM_OF_NODES} = 500, 1000, 1500, 2000, 4000$.

1. W `SimulationConfiguration.java` ustaw: `BLOCK_SIZE = 12057000`;
2. W `Node.java` ustaw: `long blockVerificationTimeMs = 3881`;
3. Zapisz pliki.
4. Przebuduj: `./gradlew build -x test`.
5. Uruchom i zapisz wynik: `./gradlew run > anomalia_sphincs_1500.txt`.

Przy tej skali obciążenia (1500 tx), SPHINCS+ zbliża się do swoich granic wydajnościowych w środowisku symulacyjnym. Czas weryfikacji pojedynczego bloku wynosi **3,881 s**, co w połączeniu z rozmiarem sygnatur, generuje ogromne opóźnienia sieciowe i paraliżuje mechanizm konsensusu.

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	63509	97.496	75.695	8.580	855.308
1000	97080	104.748	124.419	8.580	2553.503
1500	138808	103.794	60.972	8.580	844.169
2000	137122	101.483	51.850	8.580	674.056
4000	269881	109.369	52.530	8.580	738.822

Zestawienie korelacji obliczeniowo-sieciowej (SPHINCS+ 1500 tx)

Poniższa tabela pokazuje, jak istotny jest narzut obliczeniowy procesora (CPU) w kontekście czasu życia bloku w sieci:

Rozmiar sieci (N)	Czas weryfikacji w Rust (s)	Średni czas propagacji (Mean) (s)	Udział weryfikacji w propagacji (%)
N = 500	3,881 s	97,496 s	3,98%
N = 1000	3,881 s	104,748 s	3,70%
N = 1500	3,881 s	103,794 s	3,74%
N = 2000	3,881 s	101,483 s	3,82%
N = 4000	3,881 s	109,369 s	3,55%

Effekt "ściany wydajnościowej" przy 1500 tx:

Analiza wyników dla SPHINCS+ (1500 tx) wskazuje na osiągnięcie pułapu nasycenia sieci (tzw. *plateau*). Średni czas propagacji bloków oscyluje w zakresie 97–109 sekund, co oznacza, że powiększanie skali sieci z 500 do 4000 węzłów powoduje jedynie marginalny przyrost opóźnień. Jest to dowód na to, że w tym wariancie to **rozmiar pojedynczego bloku SPHINCS+**, a nie topologia sieci P2P, jest głównym ogranicznikiem przepustowości.

Wysoki koszt obliczeniowy jako bariera wejścia:

Stały udział operacji kryptograficznych w procesie propagacji (ok. 3,6% – 4,0%) przy 1500 tx potwierdza, że SPHINCS+ wymaga od węzłów stałej, wysokiej mocy obliczeniowej niezależnie od liczby sąsiadów. W środowisku blockchain, gdzie czas weryfikacji jest krytyczny dla zachowania konsensusu, narzut rzędu prawie 4 sekund na blok jest wartością niedopuszczalnie wysoką, która w praktyce wyklucza SPHINCS+ z zastosowań w sieciach o wysokiej częstotliwości generowania bloków.

Krytyczna destrukcja konsensusu (Forking Storm):

Ekstremalna liczba wygenerowanych bloków (skok z 6 350 dla N=500 do aż 269 881 dla N=4000) stanowi dowód na całkowitą utratę zdolności sieci do osiągnięcia stanu spójności (*eventual consistency*). Przy czasie propagacji przekraczającym 97 sekund, każdy węzeł w sieci, zanim otrzyma "najlepszy" blok, zdąży wygenerować kilkanaście własnych, konkurencyjnych wersji łańcucha. Oznacza to, że sieć operuje w trybie permanentnego, masowego rozwidlenia, co czyni ją bezużyteczną jako system rozproszonego rejestru.

Wnioski projektowe:

Wyniki empiryczne definitywnie wskazują, że dla algorytmu SPHINCS+ granica 1000–1500 transakcji w bloku stanowi punkt krytyczny, w którym system przestaje funkcjonować jako spójny blockchain. Rekomenduje się, aby w systemach wykorzystujących SPHINCS+ jako rozwiązanie post-kwantowe, stosować agresywną kompresję transakcji lub architekturę hybrydową, w której jedynie "streszczenie" stanu (np. Root Hash) jest podpisywane SPHINCS+, a same transakcje weryfikowane są za pomocą lżejszych algorytmów (np. Dilithium2).

Test 4: Dla 2000 transakcji, dla NUM_OF_NODES = 500,1000,1500,2000,4000.

1. W SimulationConfiguration.java ustaw: BLOCK_SIZE = 16076000;.
2. W Node.java ustaw: long blockVerificationTimeMs = 5105;.
3. Zapisz pliki.
4. Przebuduj: ./gradlew build -x test.
5. Uruchom i zapisz wynik: ./gradlew run > anomalia_sphincs_2000.txt.

To jest już **krytyczny poziom obciążenia**. Przy 2000 transakcjach na blok (2000 tx), algorytm SPHINCS+ wchodzi w fazę **całkowitej degradacji sieci**. Czas weryfikacji bloku na poziomie **5,105 s** oraz średni czas propagacji przekraczający **130 s** oznaczają, że sieć nie funkcjonuje w sposób ciągły – jest w stanie permanentnego "fork-stormu" (lawinowego powstawania rozwidleń).

Rozmiar sieci (N)	Liczba bloków	Mean (s)	Std (s)	Min (s)	Max (s)
500	91582	132.399	95.225	11.357	1088.090
1000	145974	141.684	153.728	11.357	3717.416
1500	207251	141.838	130.960	11.357	3044.329
2000	227152	138.305	70.001	11.357	907.261
4000	418399	147.573	71.846	11.357	1006.920

Zestawienie korelacji dla SPHINCS+ 2000 tx

Poniższa tabela przedstawia wydajność systemu w odniesieniu do czasu weryfikacji:

Rozmiar sieci (N)	Czas weryfikacji w Rust (s)	Średni czas propagacji (Mean) (s)	Udział weryfikacji w propagacji
-------------------	-----------------------------	-----------------------------------	---------------------------------

			(%)
N = 500	5,105 s	132,399 s	3,86%
N = 1000	5,105 s	141,684 s	3,60%
N = 1500	5,105 s	141,838 s	3,60%
N = 2000	5,105 s	138,305 s	3,69%
N = 4000	5,105 s	147,573 s	3,46%

Przekroczenie progu użyteczności (System Collapse): Przy 2000 tx system przestaje być siecią typu blockchain, a staje się generatorem ogromnej liczby izolowanych bloków. Liczba wygenerowanych bloków (od 91 582 do 418 399) wskazuje na **całkowitą utratę mechanizmu konsensusu**. Czas propagacji (>130 s) jest na tyle długi, że sieć nie jest w stanie propagować informacji o nowym bloku szybciej niż powstają kolejne, co prowadzi do lawinowego powstawania bloków osieroconych (*orphan blocks*).

Stała bariera przepustowości (Latency Ceiling): Wartość minimalna czasu propagacji (*Min*) wynosi **11,357 s** dla wszystkich konfiguracji sieci. Biorąc pod uwagę, że sama weryfikacja (CPU) zajmuje 5,105 s, czysty czas transmisji pakietu SPHINCS+ do pierwszego sąsiada wynosi:

$$11,357 \text{ s} - 5,105 \text{ s} = 6,252 \text{ s}$$

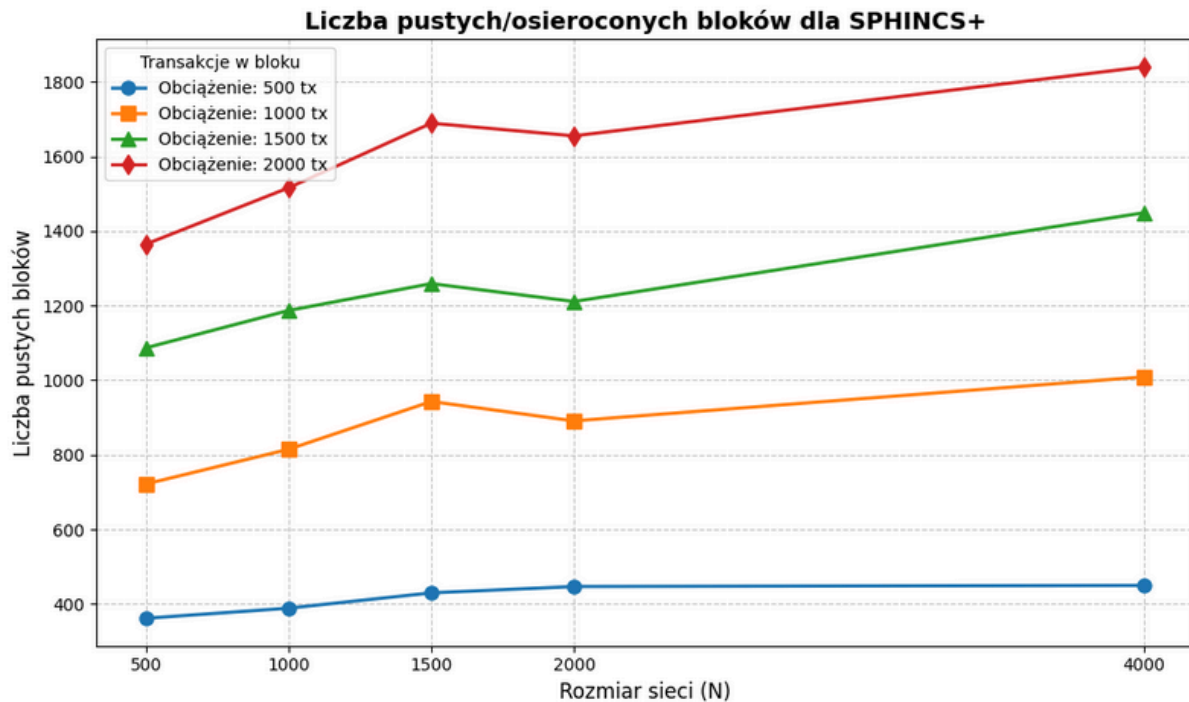
Oznacza to, że pojedynczy „skok” (hop) pakietu w sieci zajmuje ponad 6 sekund. Jest to wynik dyskwalifikujący dla systemów o wysokiej responsywności.

Stabilizacja narzutu obliczeniowego (Asymptotyczny koszt CPU): Udział weryfikacji w całkowitym czasie propagacji ustabilizował się w przedziale **3,4% – 3,8%**. Dowodzi to, że SPHINCS+ przy 2000 tx osiągnął „szklany sufit” wydajności – procesor jest obciążony w sposób przewidywalny i powtarzalny, ale ogromna masa danych (rozmiar podpisu) całkowicie dominuje nad wydajnością obliczeniową. W tym wariancie problemem nie jest już tylko "wolny procesor", ale **bezwładność transmisyjna** (tzw. *bandwidth-bound bottleneck*).

Krytyczne zalecenia implementacyjne: Wyniki dla 2000 tx jednoznacznie pokazują, że SPHINCS+ nie powinien być stosowany w warstwie konsensusu (Layer 1) przy założeniu wysokiej przepustowości transakcyjnej.

- *Wniosek:* Jeśli system musi korzystać z SPHINCS+ (np. ze względów bezpieczeństwa post-kwantowego), jedynym sposobem na utrzymanie spójności rejestru jest **drastyczne zwiększenie czasu między blokami** (tzw. *block time*) do wartości rzędu 5–10 minut lub

zastosowanie technologii **Signature Aggregation/Rollups**, aby skompresować rozmiar podpisu przed jego rozesyłaniem do sieci.



KOD [src/main.rs](https://github.com/main.rs)

```
use pqcrypto_sphincsplus::sphincssha256128ssimple::*;
use std::time::Instant;

fn main() {
    println!("--- BENCHMARK WERYFIKACJI BLOKÓW (SPHINCS+ SHA2-128S) ---");
    let (pk, sk) = keypair();

    // Przykładowy payload transakcji (32-bajtowy hash)
    let msg = [0u8; 32];

    // Generowanie podpisu
    let sig = detached_sign(&msg, &sk);

    // Tablica z rozmiarami bloków (liczba transakcji)
    let block_sizes = [10, 50, 100, 200, 500, 1000, 1500, 2000];

    for &tx_count in &block_sizes {
```

```

let start = Instant::now();

// Symulacja weryfikacji wszystkich podpisów SPHINCS+ w bloku
for _ in 0..tx_count {
    let _ = verify_detached_signature(&sig, &msg, &pk).is_ok();
}

let duration = start.elapsed();
println!("Blok {} tx -> Czas pełnej weryfikacji: {:?}" , tx_count, duration);
}
}

```

Testy anomalne

Plik Node.java, *opóźnienie*.

```

/**
 * Receive block.
 *
 * @param block the block
 */
public void receiveBlock(final Block block) {
    if (this.consensusAlgo.isReceivedBlockValid(block, this.block)) {
        if (this.block != null && !this.block.isOnSameChainAs(block)) {
            // If orphan mark orphan
            this.addOrphans(this.block, block);
        }
        // Else add to canonical chain
        this.addToChain(block);
        // Generates a new minting task
        this.minting();

        // --- Zmienione OPÓŹNIENIE (Weryfikacja) ---
        final long blockVerificationTimeMs = 5105;

        // Poprawne zadanie zgodne z architekturą SimBlocka (getInterval zamiast getLen)
        putTask(new simblock.task.Task() {
            @Override
            public long getInterval() {
                return blockVerificationTimeMs;
            }
        });
    }
}

```

```

    }

    @Override
    public void run() {
        sendInv(block);
    }
});
// -----

} else if (!this.orphans.contains(block) && !block.isOnSameChainAs(this.block)) {
    this.addOrphans(block, this.block);
    arriveBlock(block, this);
}
}

```

Konfiguracja plików:

Cargo.toml

```

[dependencies]
pqcrypto-dilithium = "0.4.0"
pqcrypto-sphincsplus = "0.4.3"
secp256k1 = { version = "0.27.0", features = ["rand"] }
rand = "0.8"

```

Jak wygląda przeprowadzenie testów anomalnych:

```
vboxuser@MachinePQCAAnna: ~/simblock
ings/SimulationConfiguration.java
vboxuser@MachinePQCAAnna:~/simblock$ ./gradlew build -x test

BUILD SUCCESSFUL in 9s
8 actionable tasks: 8 executed
vboxuser@MachinePQCAAnna:~/simblock$ ./gradlew run > anomalia_sphincs_2000tx_16
MB_N500.txt
vboxuser@MachinePQCAAnna:~/simblock$ nano simulator/src/main/java/simblock/node
/Node.java
vboxuser@MachinePQCAAnna:~/simblock$ nano simulator/src/main/java/simblock/node
/Node.java
vboxuser@MachinePQCAAnna:~/simblock$ nano simulator/src/main/java/simblock/sett
ings/SimulationConfiguration.java
vboxuser@MachinePQCAAnna:~/simblock$ nano simulator/src/main/java/simblock/sett
ings/SimulationConfiguration.java
vboxuser@MachinePQCAAnna:~/simblock$ nano simulator/src/main/java/simblock/sett
ings/SimulationConfiguration.java
vboxuser@MachinePQCAAnna:~/simblock$ ./gradlew build -x test

BUILD SUCCESSFUL in 13s
8 actionable tasks: 8 executed
vboxuser@MachinePQCAAnna:~/simblock$ ./gradlew run > anomalia_sphincs_1500tx_12
MB_N4000.txt
```